

Microcontroller Based Car Park System

Computer Science Graduation Project

Submitted By

Abdallah Salah Abdelmagied Ahmed

Youssef Ahmed Alaa-Eldin

khalid karam mohamed Soliman

Mohamed Hussein Abdallah Hassan

Mario Amir Fekry Hanna

Youssef Hamdy Amin Shamandy

Hamza Mohamed Abdallah Ebrahim

Mohamed Ayman Amin Ali Khalil

Supervised By

Prof.Dr. El Sayed A. Bakkar

T.A: Ahmed Allam

T.A: Mohraeel Kamal

Information (M.T.I) for playing a pivotal role in shaping our characters and providing us with all the necessary tools for our educational development. M.T.I. also created a wonderful environment for us to learn throughout our academic years. This accomplishment would not have been possible without the efforts of **Prof. Dr. Olfat Kamel**, President of M.T.I University, who established such a conducive learning environment, offering the best facilities for our success. It is a pleasure to thank **Prof. Dr. Mohamed Taher El-Mayah**, Dean of the Faculty of Computers & Artificial Intelligence, and **Prof. Dr. Mohamed Mohamed Elgazzar**, Vice Dean for Academic Affairs, for their unlimited help, support, encouragement, and guidance. We are honored to have worked under the supervision of **Prof. Dr. Sayed Bakar**, who has been both a mentor and a guide. We owe him a great deal for his advice and support, as well as for providing the necessary information regarding the project and following up with us through every stage until completion. It was also a pleasure to work with our Teaching Assistants **T.A. Ahmed Allam**, **T.A. Mohraeel Kamal** guided us through the learning process, never accepting less than my best efforts. We express our heartfelt thanks to both for their dedication and encouragement. Special thanks to **Prof. Dr. Hanafy Ismail**, head of the Computer Science Department, as well as the department staff, **Prof. Dr. Alaa Abd El-Raheem**, and **Prof. Dr. Mahmoud El-Shishtawy** for their efforts during our university studies. A special thank you to **Prof. Dr. Hafez Abdel-Wahab**, head of the Information System Department, and **Prof. Dr. Hesham El-Deeb**, head of the Artificial Intelligence Department, for their ongoing support throughout our academic journey. We would like to thank **Prof. Dr. Eman Taha**, the head of the Basic Science Department, and her staff, including **Prof. Dr. Rasha Saeed**, and **Dr. Rania Ahmed**, for their role in providing the foundational knowledge for our computer science degree. Special thanks go to **Prof. Dr. Anwar Assal**, head of the IT Center, for his efforts in imparting essential knowledge for our computer science degree. We are grateful to all **the Assistant Lecturers** and **Teaching Assistants** who helped us throughout our studies with practical demonstrations that deepened our understanding and provided valuable hands-on experience.

Special thanks and gratitude to our families for their unwavering support throughout our academic journey. Their consistent encouragement has been invaluable, and we recognize that our accomplishments would not have been possible without their presence and assistance

Abstract

The increasing demand for efficient, automated parking systems in Egypt environments has prompted the development of a microcontroller-based Smart Parking System. This project focuses on designing and implementing a prototype using the ESP32 microcontroller. The system aims to enhance parking management by providing real-time monitoring, automatic space detection, and user notifications.

The ESP32 microcontroller is responsible for managing sensors placed in each parking space to detect the presence of vehicles. Ultrasonic or infrared sensors are interfaced with the ESP32 to measure occupancy status, which is then transmitted to the ESP32 module. The ESP32, equipped with Wi-Fi connectivity, uploads the data to a cloud server and interfaces with a mobile application to display available parking spaces to users in real time.

Backend development incorporated the initialization and configuration of a Firebase Realtime Database, ensuring seamless communication between the mobile application and the smart home hardware. System testing was carried out to verify the functionality and reliability of the hardware features, mobile applications, and their interactions.

Key functionalities of the system include vehicle detection, database updates, user notifications, and occupancy tracking. By integrating the ESP32 sensor control, this project offers an affordable, scalable solution to streamline parking operations, reduce traffic congestion, and enhance user convenience.

Table Of Contents

1	Chapter 1 Introduction	1
1.1	Overview	1
1.2	Motivation	2
1.3	Objective	3
2	Chapter 2 Background and Literature Review	6
2.1	Introduction.....	6
2.2	Related Work.....	6
2.2.1	Abubakar Quadri, Anand Kumar, Tikesh Sahu, Preetam Kumar [1]	6
2.2.2	Ali Al Misbah, Muneer Al-Hashim. Dr. Samir Elnakla. [2]	6
2.2.3	V. Padiachy, J. Kumar, A. Chandra, K. Prakash. [3]	7
2.2.4	Fam Youssif, Fady Gamil. Prof. Dr. El Sayed A. Bakkar. [4]	8
2.3	Background Microcontroller	11
2.3.1	Types of Microcontrollers.....	11
2.4	Hardware Component	12
2.4.1	ESP32	13
2.4.2	ESP32-CAM OV2640 Camera	13
2.4.3	Servomotor.....	13
2.4.4	Sensors	14
2.4.5	Small Buzzer	14
2.4.6	A liquid-crystal display	15
2.4.7	RFID Read/Write Module RC522	15
2.4.8	Light-emitting diode	15
3	Chapter 3 System Planning and Analysis	17
3.1	Introduction.....	17
3.2	Tasks Management	17
3.3	Team Members.....	17
3.4	Task Definition:	18
3.5	Diagrams	20
3.5.1	Network Diagram.....	20
3.5.2	Task Duration	21
3.5.3	Gantt chart.....	22
3.6	System Analysis	23
3.6.1	UML Diagrams of Hardware	24
3.6.2	Application Analysis.....	28
4	Chapter 4 System Design	33

4.1	Introduction.....	33
4.2	System Block Diagram	34
4.3	System Design	35
4.4	System Components	36
4.5	Microcontroller	36
4.5.1	ESP32-CAM	37
4.6	Hardware Integration	40
4.6.1	Liquid-crystal display	40
4.6.2	RFID Read/Write Module PN532 (13.56Mhz)	41
4.6.3	Buzzer	43
4.6.4	IR Sensor	45
4.6.5	Flame Sensor	47
4.6.6	Servo motor.....	49
4.7	System Design	51
4.8	Application Design.....	52
4.9	System Scenario	53
4.10	Software Tools Used.....	54
4.10.1	Arduino IDE:	54
4.10.2	Proteus Design Suite program	55
4.10.3	Draw.io	55
4.10.4	Microsoft Word:	55
4.10.5	Visual Studio Code	56
4.10.6	Android Studio	56
5	Chapter 5 System Implementation	58
5.1	Introduction.....	58
5.1.1	Mobile App Implementation	58
5.1.2	Android (Native) Have Multiple Advantages	59
5.2	Hardware implementation.....	61
5.3	Features Implemented Using Firebase.....	62
5.3.1	Implementation Process.....	62
5.3.2	Advantages of Firebase Implementation	63
6	Chapter 6 Communication Protocols.....	65
6.1	Introduction.....	65
6.2	I2C Overview	65
6.2.2	How I2C Works	66
6.2.3	Frame Format.....	66
6.3	SPI Overview.....	66

6.3.1	Key features of SPI include	67
6.3.2	How SPI Work	67
6.3.3	Frame Format.....	69
7	Chapter 7 plate Recognition with AI.....	70
7.1	Introduction.....	70
7.2	Related Work.....	71
7.3	Tools and Technologies Used	71
7.4	YOLO V11 Overview	72
7.5	Confusion matrix.....	75
7.6	YOLO V11 Fine-Tuning	75
7.7	EasyOCR Overview.....	76
7.8	Core Architecture.....	76
7.9	EasyOCR Fine-Tuning.....	77
	Results of Fine-Tuning OCR	78
7.10	Precision & Recall & Avg Precision	78
7.11	Evaluation.....	79
7.11.1	Training and Evaluation of Yolo	79
7.11.2	Evaluation of OCR.....	80
8	Chapter 8 System Evaluation & Testing	83
8.1	Introduction.....	83
8.2	Test Case Points.....	84
9	Chapter 9 Parking Reservation Market.....	88
9.1	Introduction.....	88
9.2	Market	88
9.2.1	Target Audience.....	89
9.3	Product Positioning.....	89
9.4	Promotional Strategies	90
9.5	Conclusion.....	91
10	Chapter 10 Conclusion and future work.....	93
10.1	Conclusion.....	93
10.2	Future Work.....	94
10.3	Integration with Smart Home Systems	95
10.4	Enhanced Security Protocols.....	95
10.5	Battery Efficiency and Alternative Power Sources	95
10.6	Testing and Optimization for Diverse Environments	95
10.7	Scalability and Network Capabilities.....	95
10.8	User Interface and Experience Enhancements	96

10.9 Legal and Ethical Considerations96

11 References 97

List Of Abbreviations

AC	Alternating Current
API	Application Programming Interface
BJT	Bipolar Junction Transistor
CAM	Camera

COM	Common
CPU	central processing unit
DC	Direct Current
EEPROM	Electrically Erasable Programmable Read-Only Memory
ESP32	A series of low-cost, low-power systems on a chip microcontroller with integrated Wi-Fi and dual-mode Bluetooth
GPIO	General-Purpose Input/Output
HTTP	Hypertext Transfer Protocol
I/O	input and an output
ICSP	In Circuit Serial Programming
IDE	integrated development environment
IN	Input
IoT	Internet of Things
IR	Infrared
I2C	Inter Integrated Circuit
LCD	Liquid Crystal Displays
LED	Light Emitting Diodes
MHz	Megahertz
MISO	Master in Slave out
MOSI	Master Out Slave In
NC	normal close
NFC	Near Field Communication
NO	normal open
PCB	Printed Circuit Board
PWM	Pulse width modulation
RC522	A specific model of an RFID reader module
RFID	radio-frequency identification

SCK	Serial Clock
SDA	serial data
SDLS	Smart Door Lock System
SPI	Serial Peripheral Interface
SRAM	Static random-access memory
SSL	Secure Sockets Layer
Tinker cad	An online 3D modeling and CAD tool
UART	Universal Asynchronous Receiver-Transmitter
USB	universal serial bus
Wi-Fi	Wireless Fidelity
YOLO	You Only Look Once

List Of Figures

Figure 2-1 Related work1.....	6
Figure 2-2 Related Work2.....	6
Figure 2-3Related Work3.....	7
Figure 2-4Related Work4.....	8
Figure 2-5ESP Controller	12

Figure 2-6 Advanced ESP With Wi-Fi	12
Figure 2-7ESP-32 Controller.....	13
Figure 2-8ESPCAM	13
Figure 2-9Servo Motor	13
Figure 2-10 IR Sensor.....	14
Figure 2-11Flame Sensor	14
Figure 2-12BUZZER	15
Figure 2-13 LCD I2C.....	15
Figure 2-14RFID	15
Figure 2-15RGB LED.....	15
Figure 3-1 Network Diagram.....	20
Figure 3-2 task duration.....	22
Figure 3-3 Gannt Chart.....	23
Figure 3-4 Block Diagram	24
Figure 3-5 Flow Chart	25
Figure 3-6Context Diagram.....	26
Figure 3-7 Level 0 Diagram	27
Figure 3-8 State Machine Diagram	28
Figure 3-9 Sequence Diagram.....	29
Figure 3-10 Activity Diagram.....	30
Figure 3-11 Use Case Diagram	31
Figure 3-12 Class Diagram	32
Figure 3-13 Block Diagram	34
Figure 3-14 ESP Controller Map.....	37
Figure 3-15 ESP CAM	38
Figure 3-16 LCD With ESP	40
Figure 3-17 Buzzer with esp32	44
Figure 3-18 IR Sensor With ESP	45
Figure 3-19 Flame Sensor With ESP	47
Figure 3-20 Servo Motor With ESP.....	49
Figure 3-21 System Design	52
Figure 3-22 Application Design	53
Figure 3-23 Arduino IDE	54
Figure 3-24 Proteus Design	55
Figure 3-25 Draw.io	55
Figure 3-26 Office Applications	55
Figure 3-27 Visual Studio Code	56
Figure 3-28 Android Studio	56
Figure 5-1 Hardware Setup.....	61
Figure 5-2 Firebase Integration.....	61
Figure 6-1 I2C Protocol.....	65
Figure 6-2 I2C Frame Format	66
Figure 6-3 SPI Protocol.....	66
Figure 6-4 SPI Frame Format	69
Figure 7-1General Process.....	72
Figure 7-2 plate dataset.....	72
Figure 7-3 Yolo Architecture.....	73
Figure 7-4 compare between different versions of yolo	74

Figure 7-5 yolo Algorithm	75
Figure 7-6 Confusion matrix explanation	75
Figure 7-7 OCR Architecture.....	77
Figure 7-8 Results of fine tuning of ocr	78
Figure 7-9 Evaluation of Yolo	79
Figure 7-10 confusion matrix	80
Figure 7-11 Normalized Confusion matrix.....	81
Figure 7-12 evaluation summarized.....	82
Figure 9-1 Market Trend	88

List of Tables

Table 2-1 Related Work	9
Table 3-1 Task Definition.....	18
Table 7-1Related Work	71
Table 7-2 fine tuning Yolo	75
Table 7-3 Easy OCR for License Plate Recognition.....	77
Table 8-1 System Testing.....	84

Chapter 1

Introduction

1 Chapter 1

Introduction

1.1 Overview

The increasing economic activities in metropolitan areas have resulted in a significant rise in vehicular traffic, leading to the pressing issue of traffic congestion. One of the major challenges faced by vehicle owners in these bustling cities is the scarcity of parking spaces, particularly during peak hours. The lack of knowledge regarding the availability of parking spaces at any given time further exacerbates this problem. To address these challenges, this documentation presents an innovative.

solution an automated and smart car parking system. The proposed automated car parking system aims to alleviate metropolitan traffic congestion by providing efficient and user-friendly parking experience. By utilizing infrared transmitter-receiver pairs, the system remotely communicates the status of parking occupancy to an ESP32 controller. This information is then displayed on a prominent display board situated at the entrance of the parking lot. Through this display, vehicle owners can instantly ascertain the availability or unavailability of parking spaces before entering the parking lot, saving them valuable time and effort. The smart parking system implemented mainly in the Europe, United States and Japan (Shaheen et al., 2005) is developed with the incorporation of advanced technologies and research from various academic disciplines. With its deployment in the car park, it is hoped that it would solve the problems faced by the patrons within the car park.

The project smart car parking system is a ESP 32 microcontroller based smart parking system. Usually, in large parking areas we need to search through the whole place for space and then park the car. Our project aims to overcome this trouble. We want some sensors to be placed in space for each car. These sensors will be connected to screens like LCDs through the microcontroller at the parking entrance via microcontroller. The display would show whether space is there for more cars, and if there is any, what the exact location of that space is. This would be carried out by programming through the ESP32 microcontroller. So, the project take about general parking for all the people consist for each one has a place from which the car to enter the park and another to leave it, the output of the system is displayed on LCDs the free places for the cars that enter the park, we use for this push button switch, we will connect them to the ESP32 microcontroller to get the number of car in the park.

There will be a screen (LCD) in the place from which the car enters to show if the park is full or has places in it for another car, also show the number of empty places and which places on each floor are empty, so the person can go to any empty place and then put his car on it. On each place in each car park, we will have sensors (switch) that show if the place is empty or available, the screen of that park shows that. The system consists of a microcontroller, sensors, and displays (LCDs), all of them will be connected to each other in the project to have a completely smart car parking system

1.2 Motivation

Some challenges must always be addressed when deploying any new technology. Ensuring the security and reliability of an automatic parking system is among the most critical. Because such systems depend heavily on sensors, software, and network connectivity, any malfunction could compromise vehicle safety or facility access. Therefore, it is essential to regularly update, test, and enhance the system's hardware and software components.

The following sections will clarify the challenges during the development process

- **Power interruptions**

The parking system's sensors, gates, and guidance signage must remain operational even during grid outages. Incorporate backup power—such as UPS batteries or DC reserves—to maintain continuous functionality and avoid locking vehicles in or out.

- **Mechanical failures**

Physical components (barrier arms, tire-guidance rollers, etc.) wear overtime. Design for easy replacement of parts, use robust materials, and include self-diagnostic routines to detect and report mechanical faults before they lead to system downtime.

- **Security vulnerabilities**

Unauthorized access to the parking facility (tailgating, cloned access cards, spoofed license-plate recognition) can compromise safety. Employ strong authentication protocols, encrypted RFID or Bluetooth communications, and tamper-detection sensors to guard against intrusion.

- **User error**

Drivers may misuse payment kiosks, mistype access codes, or ignore directional signs. Simplify the user interface on kiosks and mobile apps with clear prompts, large buttons, and confirmation screens to minimize mistakes and potential lockouts.

- **Power consumption**

Sensor networks, cameras, and digital signage must balance performance with energy use, especially in off-grid or solar-powered setups. Optimize hardware sleep cycles, choose low-power components, and implement smart power-management firmware.

- **Reliability**

The system must accurately detect occupancy and process payments under varying weather

conditions, lighting levels, and vehicle types. Calibrate sensors and vision algorithms to handle rain, snow, glare, and non-standard license plates.

- **Data protection**

User data—such as payment info, license-plate images, and usage logs—must be safeguarded. Use end-to-end encryption, secure storage, and GDPR-compliant data-handling practices to preserve privacy and integrity.

- **Maintenance**

Establish a regular maintenance schedule for firmware updates, sensor calibration, mechanical inspections, and cleaning of cameras and sensors. Automated health-check reports should notify technicians before minor issues escalate.

- **Regulatory compliance**

Ensure the parking system adheres to local building codes, fire-safety regulations, and data-privacy laws (e.g., GDPR or CCPA). Factor in requirements for disabled-access signage and payment-terminal certification (e.g., PCI DSS).

- **Emergency access**

In the event of a system-wide failure or blackout, provide manual overrides: mechanical exit gates, emergency “call for help” buttons, or staff-controlled control-room switches to release barriers and guide drivers safely.

- **Programming errors**

Bugs in guidance-logic or sensor-fusion routines can lock users out or misallocate spaces. Adopt rigorous software-development best practices: code reviews, unit/integration tests, and continuous-integration pipelines to catch errors early.

1.3 Objective

This system is designed to provide a secure and smart parking environment, safeguarding vehicles. Access control mechanisms such as RFID tag readers, or license-plate recognition ensure that only authorized vehicles enter and exit the facility. By restricting access to registered users, the system helps maintain privacy and security for all parked vehicles.

Key Features of the Automatic Parking System

- **Enhanced Security**

The parking system enforces facility security through gated entry and exit points, encrypted communications, and real-time monitoring. Multiple authentication layers such as RFID tags or license-plate recognition ensure only registered vehicles can access the garage.

- **Convenience**

Users experience seamless parking through pre-registration of vehicles in a mobile or web portal. On arrival, the system automatically identifies the car (via RFID or plate recognition) and lifts the barrier without driver intervention. Payment can be handled in-app, at exit kiosks, or via automatic license-plate billing.

- **High-Accuracy Vehicle Recognition**

Advanced **computer-vision algorithms** read license plates and match them against the authorized-vehicle database. Combined with RFID verification (We use RFID in project for payment as a simulation for real payment), this dual-mode authentication greatly reduces false positives and unauthorized access attempts.

Summary

The goal of the automatic parking system is to deliver a secure, efficient, and user-friendly parking experience. By integrating multi-factor access controls, automated vehicle recognition, and real-time, the system not only heightens security but also streamlines entry, exit, and payment processes for both drivers and operators.

Chapter 2

Background and Literature Review

2 Chapter 2

Background and Literature Review

2.1 Introduction

To build a sensitive application related to the people, such as a real time parking system, it is necessary to know the components necessary to build the project from hardware parts or devices that form the basis of the system, or to identify the necessary software tools and how to use and employ them optimally, which will add to the system its distinctive features. Therefore, this chapter will discuss the components, tools, experiences, and previous experiences in building a real time parking system.

2.2 Related Work

2.2.1 Abubakar Quadri, Anand Kumar, Tikesh Sahu, Preetam Kumar [1]

In this system describes a Car Parking Control System is designed to efficiently manage parking spaces in crowded areas. It automates parking operations using sensors, microcontrollers, and LED indicators to guide drivers to available slots. The system provides real-time updates, enhances user convenience,

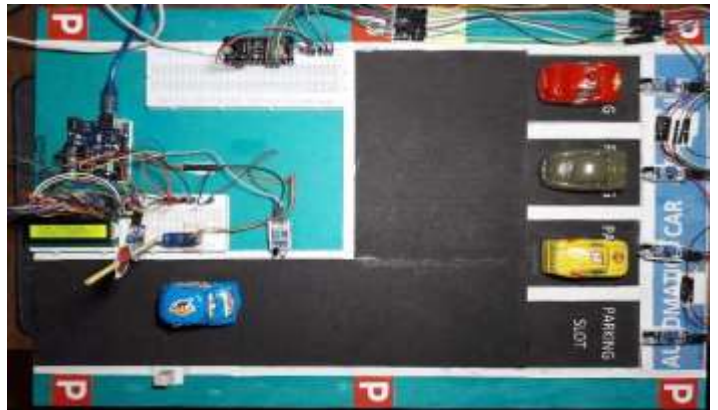


Figure 2-1 Related work1

and minimizes parking mishaps. Compact and modern, it offers scalability and additional features like password protection and time-based billing. Microcontroller: PIC16F877A for system control Sensors: Infrared (IR) sensors for vehicle detection. LCD: To display available parking slots and messages. Barrier System: Operated by a DC motor with a circuit controller. LED Indicators: Green LEDs to mark available parking spaces. PS/2 Keyboard: For user input and querying parking information. Alarm System: To alert people about unauthorized parking or errors.

2.2.2 Ali Al Misbah, Muneer Al-Hashim. Dr. Samir Elnakla. [2]

In our study, we explored the realm of the parking system. The Parking Monitoring and Control System successfully addresses the challenges of parking management by integrating efficient sensors, an Arduino microcontroller,



Figure 2-2 Related Work2

and a database system. It provides real-time data on parking space availability, enhances security, and minimizes environmental impact. The system is cost-effective, reliable, and user-friendly, making it suitable for high-traffic areas. With potential for future enhancements, such as increased automation and scalability, the project demonstrates a practical and impactful solution for modern parking management needs.

Arduino Mega 2560 – A microcontroller that processes input and controls output devices.

RFID Reader and Tags – For identifying authorized personnel and logging entry/exit data.

Infrared (IR) Sensors – To detect vehicle presence and assist in gate automation.

16x24 LED Matrix Display – Displays the number of available parking spaces and alerts.

Servo Motors – Control the opening and closing of gates.

Power Supply Unit – Provides 9V DC for Arduino and 5V DC for other components.

PC with C# Software – Handles database management and user interface functionalities.

2.2.3 V. Padiachy, J. Kumar, A. Chandra, K. Prakash. [3]

This project describes an automated multi-level car parking system to optimize land usage and address parking shortages. The system incorporates automated mechanisms to lift, move, and position vehicles in designated slots across multiple levels. It reduces manual intervention, enhances operational efficiency, and minimizes land and construction costs. The prototype employs mechatronic principles, focusing on automation through sensors, actuators, and microcontrollers.



Figure 2-3 Related Work3

Light-Dependent Resistors (LDRs) – To detect slot occupancy.

DC Motors – For X, Y, and Z movements of the parking mechanism.

Lead Screw Mechanism – For horizontal motion.

Belt Drive System – For vertical motion.

PIC Microcontroller (PIC18F8722) – For controlling motor operations.

2.2.4 Fam Youssif, Fady Gamil. Prof. Dr. El Sayed A. Bakkar. [4]

The project focuses on an automated parking gate control system using a microcontroller, designed to tackle parking challenges in metropolitan areas. The system employs infrared sensors to detect vehicle entry and exit, displaying parking slot availability on an LCD screen at the entrance. A web application complements the hardware by providing real-time updates about parking occupancy. This solution aims to reduce congestion, save time, and improve overall parking efficiency.

Arduino Mega 2560 – Main microcontroller. Infrared (IR) Sensors – To detect vehicle presence in parking slots. LCD Display (20x4) – For showing parking slot availability. Servo Motor – For opening and closing the parking gate. LEDs – To indicate slot availability. Buzzer – For audio alerts. Node MCU – For IoT connectivity. Fire Alarm Sensor – To detect potential hazards.



Figure 2-4Related Work4

Table represents the Literature Review as

Title & year	Authors	Advantages	Disadvantages	Methodology
IoT Based Car Parking Manager (2022)	Abubakar Quadri, Anand Kumar, Tikesh Sahu, Preetam Kumar	Automates the toll collection process, reducing manual work and inefficiency. Real-time Parking Availability. Reduces traffic jams and fuel waste.	Maintenance Costs Parking contains one gate that may cause a collision.	The car parking system is a novel solution designed to provide a convenient and efficient way for drivers to find available parking spots.
IoT Based Car Parking Manager (2013)	Ali Al Misbah, Muneer Al-Hashim, Salman Al-Belaihi, Dr. Samir Elnakla	The system contains two IR Sensors for each Gate (Enter or Exit) LCD for display Empty and full places in parking. Two gates. System control uses app. The system uses RFID technology.	There are problems that occur because of the misuse of the software by the staff.	The system uses IoT and RFID technology to automate toll collection. A microcontroller-based system reads the RFID cards of vehicles arriving at toll booths. The system checks the card's validity and balance via an online server. Once the balance is confirmed, the toll amount is deducted, and the toll gate. The vehicle data and toll transaction information are stored in a database for later reference.
IOT Based multilevel Car	V. Padiachy, J. Kumar, A. Chandra, K. Prakash	Parking will carry many cars rather than traditional parking.	High cost because material that is used in this project. The system will cause much wasted time to park cars or take cars from parking.	The IoT-based car parking system is a cutting-edge technology solution designed to revolutionize the way we park our vehicles in urban areas. The system utilizes a network of sensors, servo motors,

Parking System (2022)			System may fail and car damaged	and Wi-Fi modules to detect and manage parking availability in real-time, providing users with a convenient and hassle-free parking experience.
Smart Parking System Based on IoT (2023)	Fady Gamil Abdelmassih, Fam Youssef Fouad, Reham Eid Mohamed	Automates the toll collection process, reducing manual work and inefficiency. Real-time Parking Availability Reduces traffic jams and fuel waste. system works with websites.	Web Application does not contain many features. System does not contain Alarm System. Parking contains only one gate that causes collusion.	The IoT-based car parking system is a novel solution designed to provide a convenient and efficient way for drivers to find available parking spots in urban areas. The system utilizes Internet of Things (IoT) technology, including sensors, servo motors, and Wi-Fi modules, to detect and manage parking availability in real-time.

2.3 Background Microcontroller

A microcontroller is a small computer on a single integrated circuit that is designed to control specific tasks within electronic systems. It combines the functions of a central processing unit (CPU), memory, and input/output interfaces, all on a single chip.

Microcontrollers are widely used in embedded systems, such as home appliances, automotive systems, medical devices, and industrial control systems. They are also used in consumer electronics products, such as gaming systems, digital cameras, and audio players. The programming languages used to write code for microcontrollers vary depending on the manufacturer and the type of microcontroller. Some of the commonly used programming languages include C, C++, and assembly language.

2.3.1 Types of Microcontrollers

- **ARM Microcontrollers:** These microcontrollers are based on the ARM architecture and are widely used in a variety of applications, including mobile devices, automotive systems, and industrial control systems.
- **PIC Microcontrollers:** These microcontrollers are manufactured by Microchip Technology and are commonly used in a wide range of applications, including home appliances, smart door lock, automotive systems, and medical devices.
- **AVR Microcontrollers:** These microcontrollers are manufactured by Atmel Corporation and are commonly used in applications such as robotics, industrial control systems, smart door lock, parking system, and consumer electronics.

So, there are many manufacturers of microcontrollers in the world of technology, and here is a group of the largest manufacturers and the most famous microcontrollers that may be suitable for use in automated parking systems.

2.3.1.1 ESP microcontroller Overview

ESP refers specifically to a series of microcontrollers developed by Expressive Systems, a Chinese manufacturer. These are popular microcontrollers for hobbyists and professionals due to their affordability, features, and ease of use. There are different ESP microcontrollers available, each with varying capabilities. Some popular options **include**:

- **ESP32:**

This is a powerful and versatile option with a dual core processor, Wi-Fi, Bluetooth, and a good number of input/output (I/O) pins. The ESP32 is the successor to the ESP8266. It adds extra CPU cores, faster Wi-Fi, more GPIO, and supports Bluetooth.



Figure 2-5 ESP Controller

- **ESP8266:**

The ESP8266 is an excellent choice for beginners and experienced makers alike due to its affordability, ease of use, and Wi-Fi capabilities. It opens a world of possibilities for creating internet-connected devices and fun IoT projects.



Figure 2-6 Advanced ESP With Wi-Fi

2.4 Hardware Component

2.4.1 ESP32

This is a powerful and versatile option with a dual core processor, Wi-Fi, Bluetooth, and a good number of input/output (I/O) pins. The ESP32 is the successor to the ESP8266. It adds extra CPU cores, faster Wi-Fi, more GPIO, and supports Bluetooth.



Figure 2-7ESP-32 Controller

2.4.2 ESP32-CAM OV2640 Camera

The ESP32-CAM is a small size, low power consumption camera module based on ESP32 WIFI development board. It comes with an OV2640 camera, and it also features a microSD card slot that can be useful to store images taken with the camera or to store files to serve to clients.



Figure 2-8ESPCAM

2.4.3 Servomotor

A servo motor is a rotary or linear actuator designed for precise control of angular or linear position, speed, and acceleration. It operates by receiving a control signal that specifies the desired position, then adjusts its output using feedback mechanisms to maintain accuracy. Servo motors typically consist of a DC or AC motor.

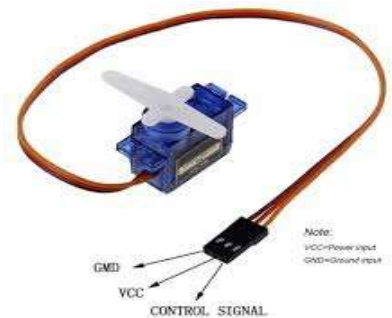


Figure 2-9Servo Motor

2.4.4 Sensors

A sensor is a device that detects physical, chemical, or environmental changes and converts them into electrical signals for analysis or control. Sensors are integral to modern systems, enabling interaction with the physical world by measuring variables like temperature, pressure, motion, light, or sound. They are widely used in applications such as automation, healthcare, IoT, and robotics. Advanced sensors often include calibration and processing capabilities for improved accuracy and performance

2.4.4.1 IR Infrared Sensor

IR Sensor is an electronic device that emits infrared lights to sense some aspect of the surroundings and can be employed to detect the motion of an object. IR sensor module is very simple it consists of two main components: the first is the IR transmitter section and the second is the IR receiver section.



Figure 2-10 IR Sensor

2.4.4.2 Flame Sensor

A relay is an electrically operated switch. The nature of a relay in Arduino is to act as a switch that can be controlled electronically. It allows the ESP32 to control higher voltage or current devices than the ESP32 can handle directly.

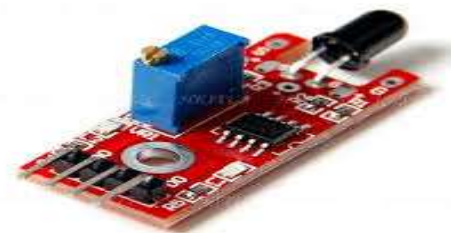


Figure 2-11 Flame Sensor

2.4.5 Small Buzzer

Buzzer means electronic component that generates sound through the transmission of electrical signals. Its primary function is to provide



an audible alert or notification and typically operates within a voltage range of 5V to 12V

Figure 2-12BUZZER

2.4.6 A liquid-crystal display

Liquid Crystal Displays (LCDs) are a vital component in many embedded systems, LCDs provide a user-friendly way to interact with the embedded system, displaying information, menus, and prompts for user input. offering a valuable way to visualize information and create user-friendly interfaces. Their versatility, low power consumption, and various interface options make them a popular choice for embedded system developers. LCD display [16x2, 16x4, 20x2 or 20x4 LCDs] is used for indicating the system status / parameters.



Figure 2-13 LCD I2C

2.4.7 RFID Read/Write Module RC522

RFID means radio-frequency identification. refers to a wireless system comprised of two components: tags and readers, RFID uses electromagnetic fields to transfer data over short distances. RFID is useful to identify people, to make transactions, etc. You can use an RFID system to open a door. For example, only the person with the right information on his card is allowed to exist.



Figure 2-14RFID

2.4.8 Light-emitting diode

LEDs (Light Emitting Diodes) are fundamental components commonly used in embedded systems for various purposes. Here's how LEDs play a crucial role in these tiny computers: Status Indicators, Controlling Brightness. Overall, LEDs are a versatile and beginner friendly component that plays a significant role in the world of embedded systems. They provide valuable tools for



Figure 2-15RGB LED

indication, debugging, prototyping, and even simple output control in various projects.

Chapter 3

System Planning and Analysis

3 Chapter 3

System Planning and Analysis

3.1 Introduction

This chapter emphasizes the critical role of meticulous planning in building a smart and reliable automatic parking system. this system aims to Recognizing the importance of user safety, reliability and scalability. This chapter presents the project goals and clarifies preciously the requirements which are needed to perform the project planning. The project plan includes It then goes deep into the process of breaking down the project into manageable tasks, enhancing efficient distribution among team members. Furthermore, the chapter outlines the creation of a project timeline, ensuring a well-defined schedule for development. It sheds light on the basics and components of the system, along with their important roles in achieving overall functionality. By emphasizing the importance of planning and outlining its key elements, this chapter sheds light on the foundation for a successful and reliable smart parking system. This chapter goes deep into the project planning specifics, focusing on effective team collaboration and a well-defined development timeline addresses the crucial aspect of task division among team members. The Gantt chart, a visual representation of the project timeline. Finally, the network diagram. By outlining these elements, the chapter emphasizes the importance of careful planning in achieving a successful and efficient smart parking system development process.

3.2 Tasks Management

This section highlights the importance of teamwork in constructing a reliable smart parking system. Recognizing the value of collaboration, it outlines the project team structure, comprised of seven individuals with specialized tasks. By fostering a collaborative environment and leveraging diverse skillsets, this approach aims to optimize the smart parking system's development process and achieve project success.

3.3 Team Members

- 1) Abdallah Salah Abdelmagid Ahmed

- 2) Youssef Ahmed Alaaeldin
- 3) Khaled Karim Mohamed Soliman
- 4) Mohamed Hussein Abdallah Hassan
- 5) Mario Amir Fekry Hanna
- 6) Youssef Hamdy Amin Shamandy
- 7) Hamza Mohamed Abdallah Ebrahim
- 8) Mohamed Ayman Amin Ali Khalil

3.4 Task

Definition:

This section provides a breakdown of the team roles and tasks

involved in building the automatic parking system.

Table 3-1 Task Definition

Development Life Cycle Phases	Student Participates
Problem Definition & Literature Survey	All
System Analysis: - Collect Data - System Software and Simulation	All
Identify Application Architecture: - Design App - Signal Flow Diagram - Study the Required Code	Abdallah salah, Youssef Ahmed, Khaled Kariem, Mario Amir, Mohamed Hassan
Identify the Hardware Components: - Draw Prototype of Circuit Diagram. - Signal Flow Diagram. - Study the Required Code.	All
Identify The Ai Model Architecture - Collect Dataset - Study The Required Code	Abdallah, Youssef Ahmed
System Implementation: System Implementation with Esp32: - Components Integration - Coding	Abdallah, Youssef Ahmed, Khaled, Mohamed Hassan
System Implementation with App: Coding	Abdallah Salah, Youssef Ahmed, Khalid Kariem, Mario Amir
System Implementation with Ai Model Coding	Abdallah, Youssef Ahmed
System Implementation with Esp32-Cam: - Components Integration - Coding	Abdallah, Youssef Ahmed, Khaled

System Implementation with Esp32, Esp32-cam, App, Ai Model: • Components Integration • Coding	All
System Testing	All
System Documentation: • System Documentation • System Presentation	All

3.5 Diagrams

3.5.1 Network Diagram

This section presents a critical project management tool - the network diagram visually depicts the project (tasks dependence).

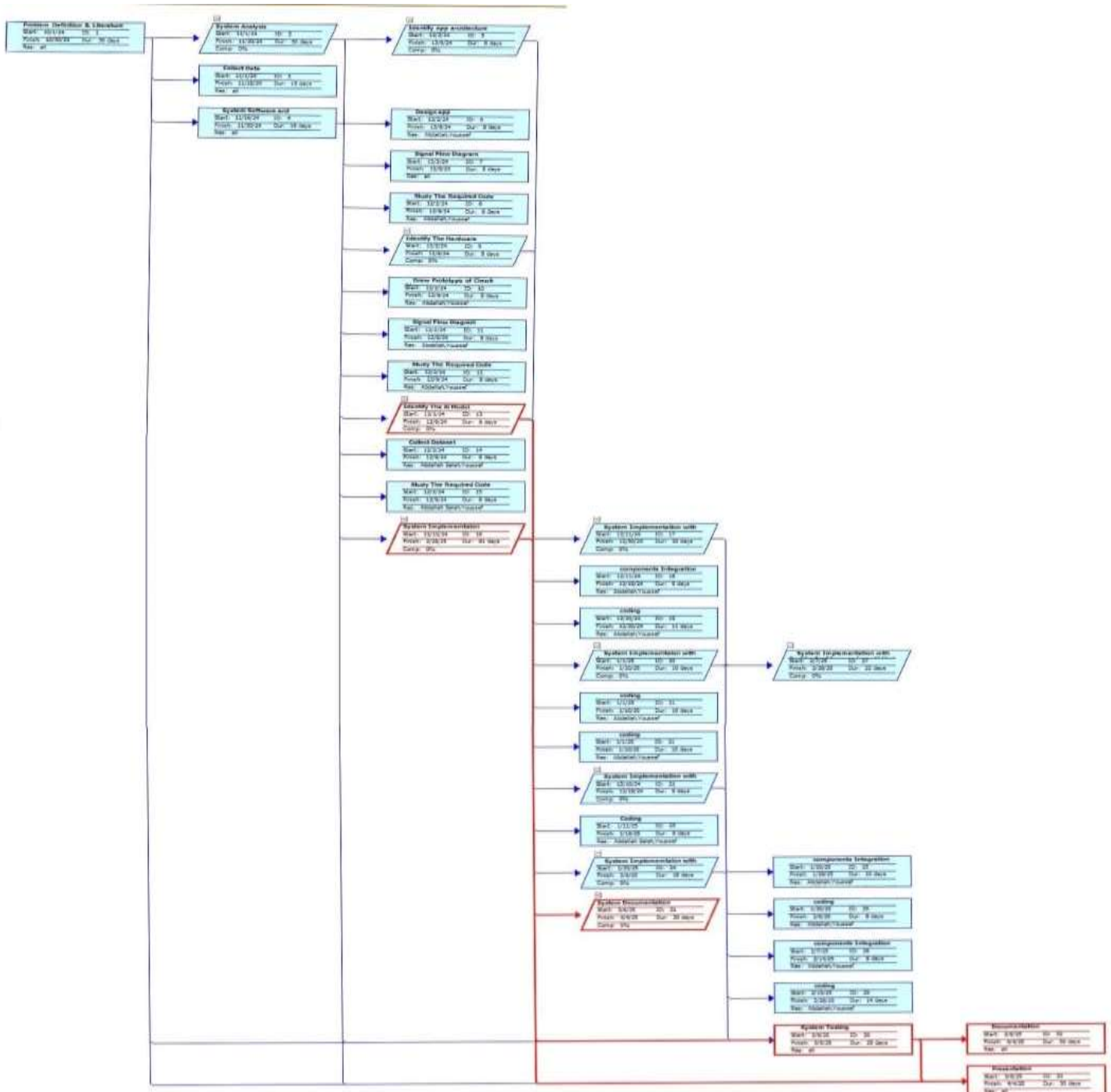


Figure 3-1 Network Diagram

3.5.2 Task Duration

This section presents the project schedule visualized through Gantt charts, a popular project management tool commonly used in project management, is one of the most popular and useful ways of showing activities (tasks or events) displayed against time. Each activity is represented by a bar; the position and length of the bar reflects the start date, duration, and end date of the activity the distribution of the tasks in the project on the timeline is represented as Gantt Chart in figure below.

	Task Name	Duration	Start	Finish	Predecessors	Resource Names
1	Problem Definition & Literature Survey	30 days	10/1/2024	10/31/2024		All
2	System Analysis	30 days	11/1/2024	12/1/2024	1	All
3	Collect Data	15 days	11/1/2024	11/15/2024	1	All
4	System Software and Simulation	15 days	11/16/2024	12/1/2024	2	All
5	Identify App Architecture	8 days	12/2/2024	12/10/2024	4	All
6	Design app	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
7	Signal Flow Diagram	8 days	12/2/2024	12/10/2024	4	All
8	Study The Required Code	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
9	Identify The Hardware components	8 days	12/2/2024	12/10/2024	4	All
10	Draw Prototype of Circuit Diagram	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
11	Signal Flow Diagram	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed Mohamed Amin Mohamed Hassam Youssef Hamdi
12	Study The Required Code	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
13	Identify The Ai Model Architecture	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed
14	Collect Dataset	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed
15	Study The Required Code	8 days	12/2/2024	12/10/2024	4	Abdallah Salah Youssef Ahmed
16	System Implementation	79 days	12/11/2024	3/3/2025	5,9,13	All
17	System Implementation with Esp32	20 days	12/11/2024	12/31/2024	9	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
18	Components Integration	9 days	12/11/2024	12/20/2024	9	Abdallah Salah Youssef Ahmed Mohamed Amin Mohamed Hassam Youssef Hamdi
19	Coding	11 days	12/21/2024	12/31/2024	9	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
20	System Implementation with App	10 days	1/1/2025	1/11/2025	5	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir

	Task Name	Duration	Start	Finish	Predecessors	Resource Names
21	Coding	10 days	1/1/2025	1/11/2025	5	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
22	System Implementation with Ai Model	9 days	1/11/2025	1/20/2025	13	Abdallah Salah Youssef Ahmed
23	Coding	9 days	1/11/2025	1/20/2025	13	Abdallah Salah Youssef Ahmed
24	System Implementation with Esp32-cam	18 days	1/21/2025	2/8/2025	17,20	All
25	Components Integration	10 days	1/21/2025	1/31/2025	17,20	Abdallah Salah Youssef Ahmed Mohamed Amin Mohamed Hassam Youssef Hamdi
26	Coding	8 days	2/1/2025	2/8/2025	17,20	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
27	System Implementation with Esp32, Esp32-cam, App, Ai Model	22 days	2/9/2025	3/3/2025	17,20,22,24	All
28	Components Integration	8 days	2/9/2025	2/17/2025	17,20,22,24	Abdallah Salah Youssef Ahmed Mohamed Amin Mohamed Hassam Youssef Hamdi
29	Coding	14 days	2/18/2025	3/3/2025	17,20,22,24	Abdallah Salah Youssef Ahmed Khalid Kariem Mario Amir
30	System Testing	25 days	3/4/2025	3/29/2025	16	All
31	System Documentation	30 days	3/30/2025	4/30/2025	1,2,5,9,13,16,30	All
32	Documentation	30 days	3/30/2025	4/30/2025	1,2,5,9,13,16,30	All
33	Presentation	30 days	3/30/2025	4/30/2025	1,2,5,9,13,16,30	All

Figure 3-2 task duration.

3.5.3 Gantt chart

in the design and making necessary adjustments, a thorough system analysis can optimize the performance of the system.

3.6.1 UML Diagrams of Hardware

3.6.1.1 System Block Diagram

depicts the project's system block diagram. It illustrates the major components and their interactions at a high level, providing a clear understanding of the system's overall flow.

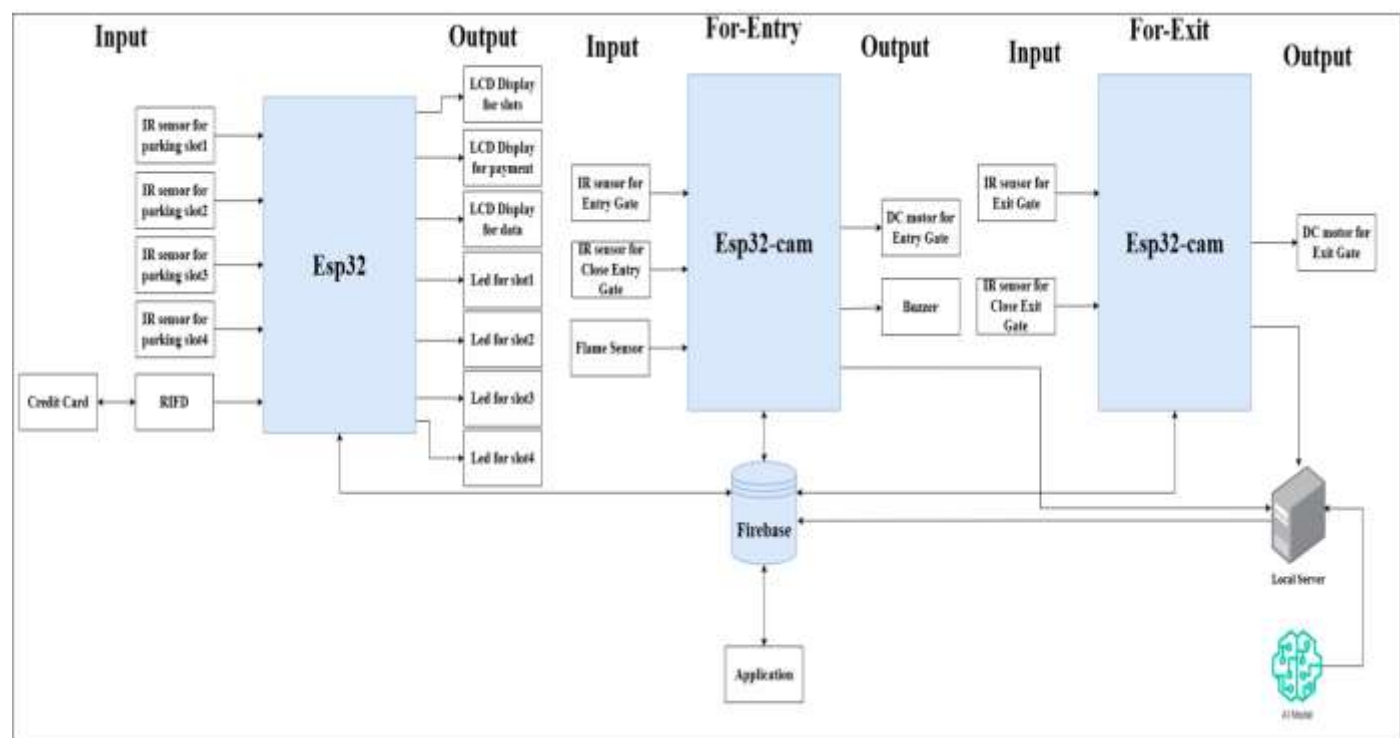


Figure 3-4 Block Diagram

3.6.1.2 Flow Chart Diagram

The below shown represents the project’s flow Chart diagram simply representing the logical sequence in an easy manner and language making it easy to understand and feasible to fathom.



Figure 3-5 Flow Chart

3.6.1.3 Context Diagram

Figure 3.6 depicts the project's context diagram. This high-level view focuses on the system's interactions with external entities, providing a clear understanding of the system's boundaries and the data or information that flows between them. This simple representation helps establish the scope of the project and facilitates communication among stakeholders.

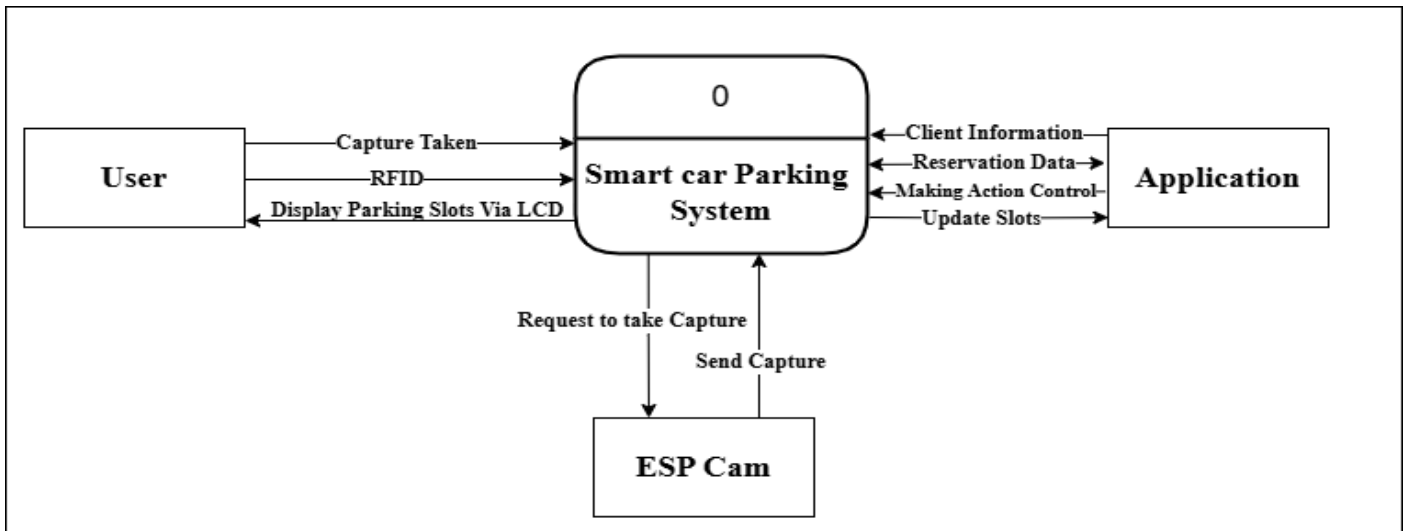


Figure 3-6Context Diagram

3.6.1.4 LEVEL 0 Diagram

The figure below shown represents the project's level 0 diagram. This high-level view focuses on the system's interactions with external entities, providing a clear understanding of the system's boundaries and the data or information that flows between them. This simple representation helps establish the scope of the project and facilitates communication among stakeholders

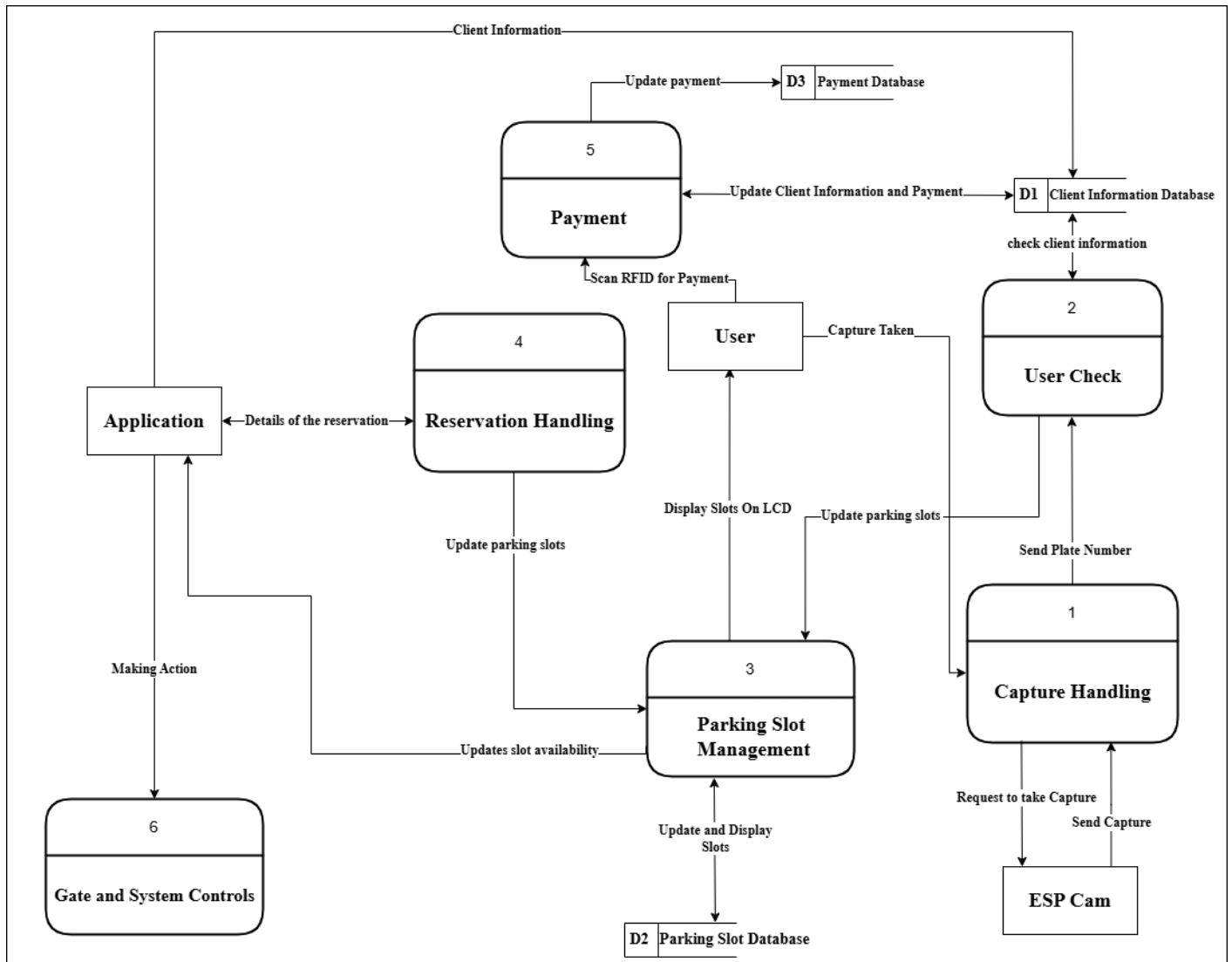


Figure 3-7 Level 0 Diagram

3.6.1.5 State Machine Diagram

The state machine diagram, illustrated below, depicts the system's behavior in a clear and understandable manner, much like a flowchart diagram. It focuses on the various states the system can occupy and the events that trigger transitions between these states. This visual representation makes it easy to follow the logical sequence of the system's behaviors under different conditions.

This description highlights the key aspects of a state machine diagram:

States: Like flowchart steps, the diagram shows the different states the system can be in.

Events: These represent the triggers that cause the system to transition from one state to another.

Clarity: Just like a flowchart, the focus is on presenting the information in an easy-to-understanding way.

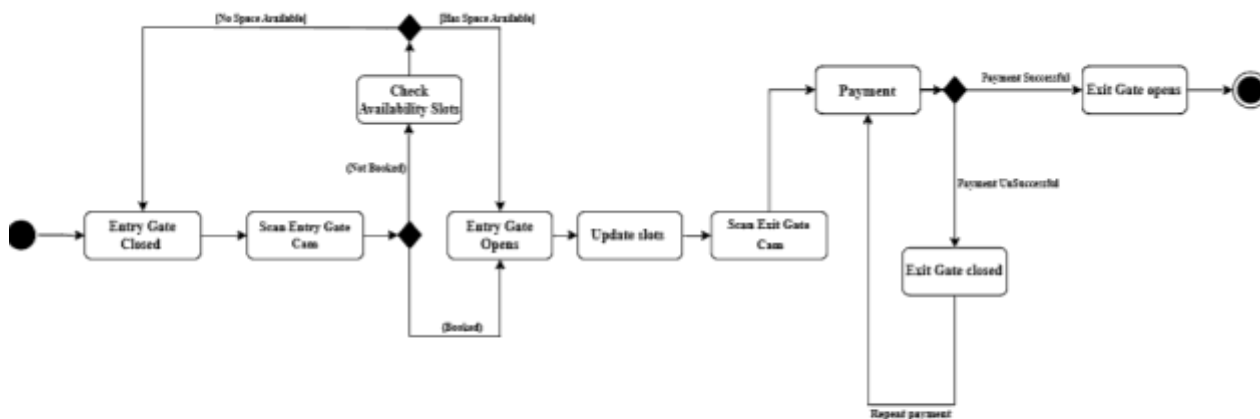


Figure 3-8 State Machine Diagram

3.6.2 Application Analysis

3.6.2.1 Sequence Diagram

This sequence diagram illustrates the interactions between a User and a Parking Reservation System to perform login/signup, view available parking slots, and reserve a slot. The system interacts with a Database to manage user and reservation data.

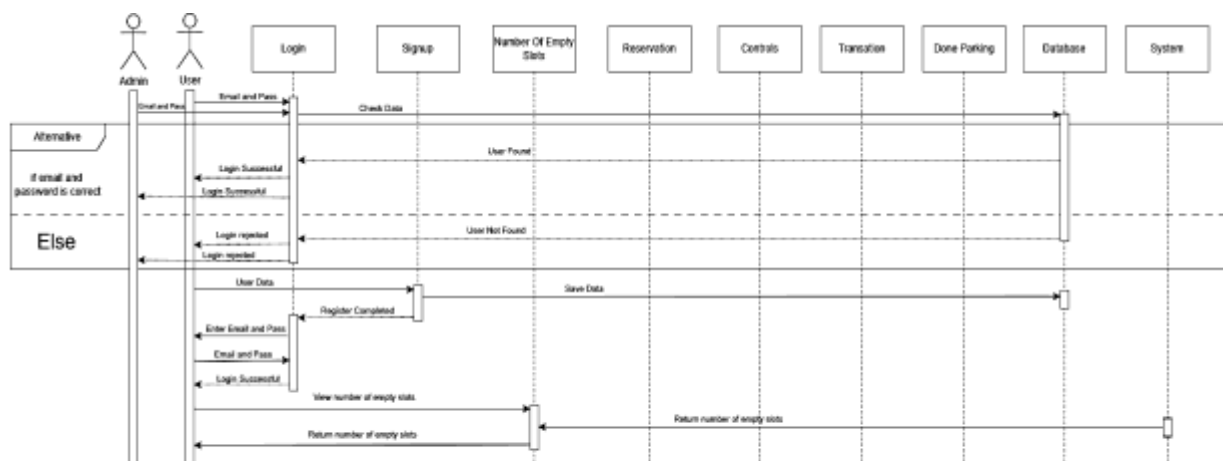


Figure 3-9 Sequence Diagram

3.6.2.2 Activity Diagram

The activity diagram outlines the user's journey through the parking reservation system. It shows that users can either log in with existing credentials or sign up for a new account before proceeding to reserve a parking slot, provide necessary reservation details, and complete the process by selecting a payment method.

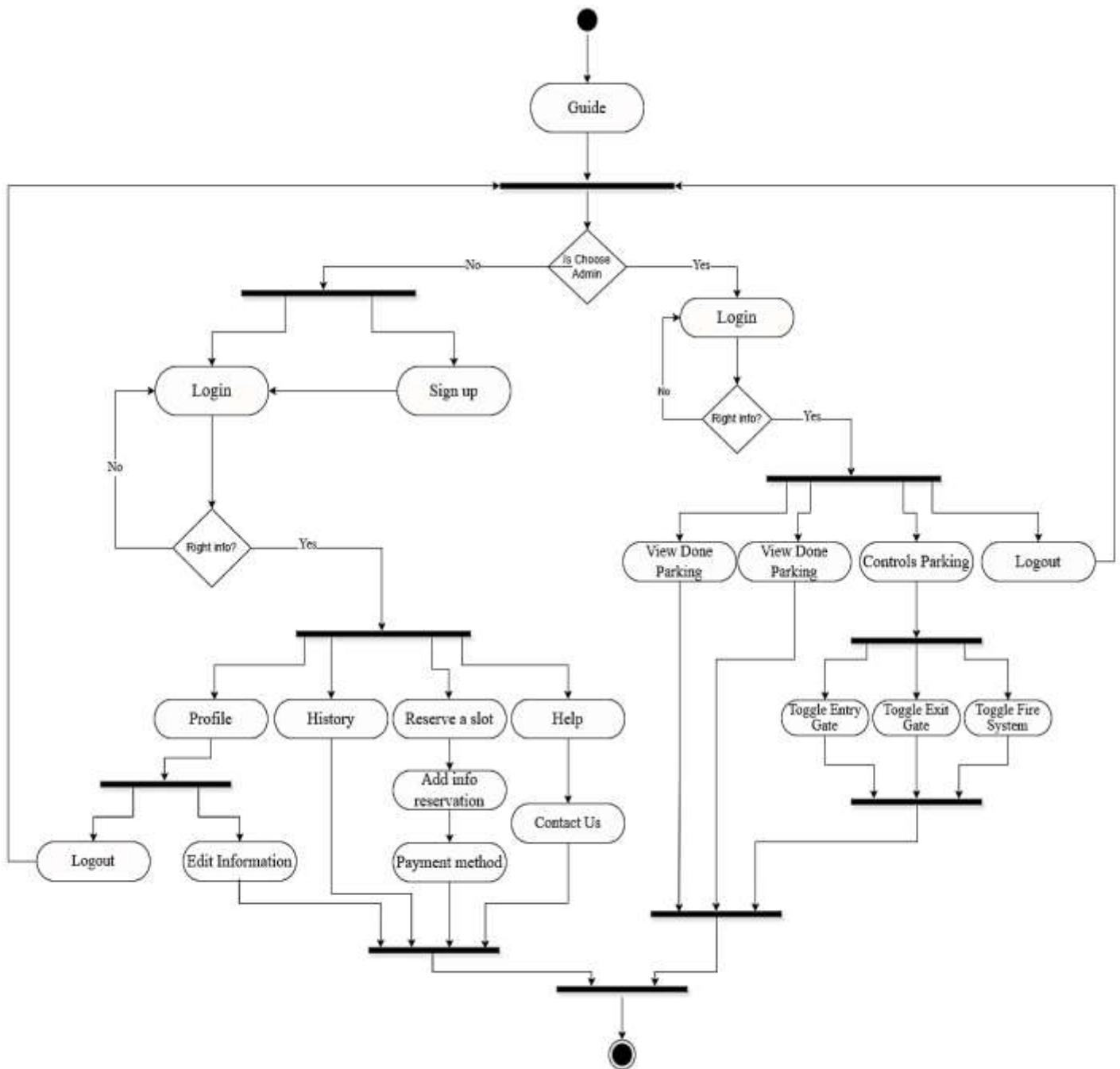


Figure 3-10 Activity Diagram

3.6.2.3 Use case Diagram

The use case diagram provides a high-level overview of the car parking system's functionalities. It highlights how users can log in/sign up and book parking slots, while administrators have control over slot availability, alarms, and gates. The Server plays a crucial role in managing data and handling core system functionalities like authentication and booking confirmation.

Figure 3-11 Use Case Diagram

3.6.2.4 Class Diagram

This class diagram models the core components of a parking reservation system. It shows how Users and Admins interact with the system. Users can log in/sign up, view available parking Slots, and create or cancel Bookings. The system uses Notifications to communicate with users and Alarms to signal events to the Admin. The Booking class manages reservation details and its status. All persistent data is managed by the Database class, which provides operations for saving, updating, finding, and deleting information. The

relationships between these classes define how they interact and depend on each other to provide the functionality of the parking reservation system.

Figure 3-12 Class Diagram

Chapter 4

System Design

4 Chapter 4

System Design

4.1 Introduction

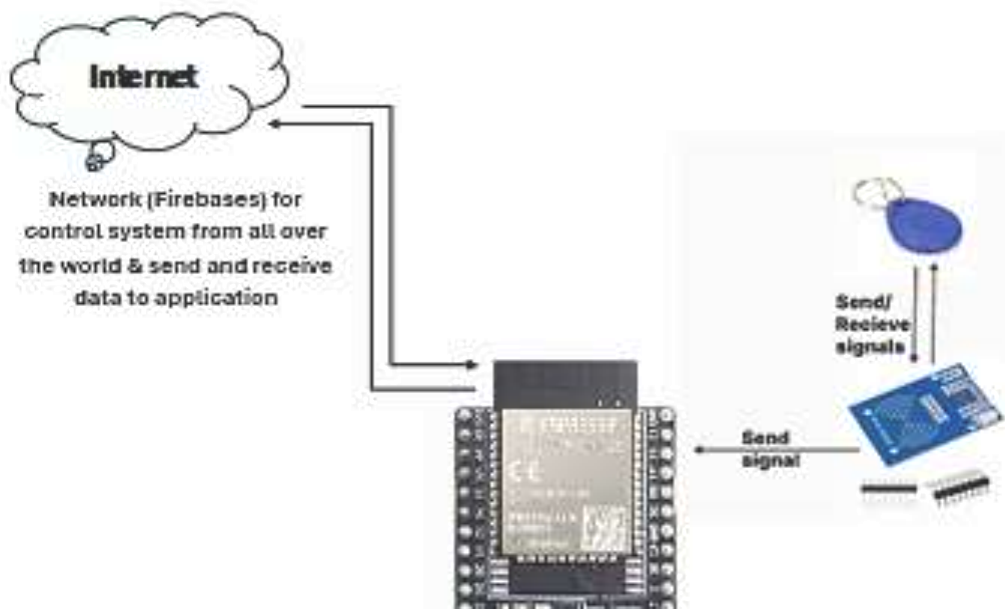
This chapter delves into the practical construction of a **smart parking system**, building on the basic principles established previously. It explains the step-by-step process and explains the system architecture in figure 4.1. This section focuses on the three basic modules - the parking space/barrier, the user/vehicle, and the central management system, and explains their distinct roles and seamless interaction to create a

unified system for efficient parking management in locations like 6th of October City, Giza Governorate, Egypt. Sections 4.5 talk about the components that will be used, such as parking sensors, cameras, and communication modules. Section 4.7 delves into the hardware integration of the parking infrastructure itself, detailing how the hardware components such as the parking barriers, vehicle detection sensors, and potentially license plate recognition technology work together. Finally, Section 4.11 explains the software tools used for programming the embedded systems and developing the user interface for the smart parking application. In the project

4.2 System Block Diagram

The function block diagram as shown in figure 4.1 represents a visual observation of the smart parking system. A smart parking system is a technology-driven solution that utilizes electronic components and network connectivity to optimize the utilization and management of parking spaces. It typically replaces traditional manual parking management methods and provides a more convenient and efficient way for drivers to find and access parking in locations like 6th of October City, Giza Governorate, Egypt

Block Diagram



4.3 System Design

We will dissect the key components, the microcontroller, the sensors, and the actuators and explore how they work together to create a secure and user-friendly entry system. We'll delve into the decision-making process behind the microcontroller, the data gathering power of the sensors, and the physical actions carried out by the actuators.

4.4 System Components

To achieve the required functionality of this project, the components were chosen based on a certain criterion, starting with the main component (the microcontroller) then the other inputs and outputs.

4.5 Microcontroller

The ESP32-WROOM is a specific module variant within the ESP32 family of microcontrollers made by Expressive Systems, designed for Wi-Fi and Bluetooth connectivity in IoT applications. The ESP32 chip integrates a dual-core processor with 448 Kbyte ROM, 520 Kbyte SRAM, 16 Kbyte SRAM in RTC, 802.11 b/g/e/I Wi-Fi, Bluetooth v4.2 BR/EDR & BLE, clocks & Times, abundant peripheral Interfaces and security mechanism. Software can be developed using many different platforms, including Arduino. It is designed for Generic low power IoT sensor hub, loggers, video steaming for camera, Wi-Fi & Bluetooth enabled devices, home automation and mesh network applications, aimed at makers, hardware engineers, software engineers and solution providers.

Some Of the Key Features

- CPU and Memory: Xtensa® LX6 Dual-core processor, up to DMIPS
- Chip ESP32-D0WD
- 4 MByte(32Mbit) SPI Flash



32-bit
600

- 448 KByte ROM
- 520 KByte SRAM
- 16 KByte SRAM in RTC
- PCB Antenna Connector
- Supply Voltage: 2.2V~3.6V

Figure 3-14 ESP Controller Map

ESP Microcontroller are a popular

platform for hobbyists and professionals alike to develop electronic projects. They are single-board computers designed for ease of use and rapid prototyping. Here's a breakdown Using Arduino with Arduino IDE:

- **Choose Your Board:** Select the specific ESP board you'll be using in the IDE.
- **Write Your Code:** Use the Arduino IDE to write your code, including instructions for the microcontroller to control various components connected to it.
- **Compile and Upload:** Compile your code to convert it into a format the ESP board understands. Then, upload the compiled code to your board using the IDE.
- **Connect Components:** Connect sensors, actuators, and other electronic components to the Arduino board's I/O pins following the specific requirements of your project.

Installing the Arduino IDE is a straightforward process. Here's how to do it on Windows, macOS, and Linux:

- Download the Arduino IDE software from the official website.
- Double-click the downloaded .exe file to start the installer.
- Follow the on-screen instructions in the installer wizard. It's typically recommended to keep the default settings during installation.
- Once the installation is complete, you can launch the Arduino IDE from your Start menu.

4.5.1 ESP32-CAM

The ESP32-CAM is a specific module variant within the ESP32 family of microcontrollers made by Espressif Systems, uniquely equipped with a camera for visual data acquisition in IoT applications, particularly relevant for a smart parking system in 6th of October City, Giza Governorate, Egypt. The ESP32 chip at its core integrates a dual-core processor with ample memory resources and comprehensive connectivity options including Wi-Fi and Bluetooth, enabling it to **capture images**, process them

(potentially with onboard capabilities), and transmit data wirelessly. Its features make it ideal for tasks such as vehicle detection, parking spot occupancy monitoring, and even **license plate recognition when integrated with appropriate software and AI models**. Software development for the ESP32-CAM can be done using platforms like Arduino, offering ease of use for implementing image capture and communication functionalities for the smart parking project.

Component

- ESP32-CAM module (with integrated camera).
- USB cable for power and programming.
- Potentially mounting hardware for optimal camera positioning

Figure 3-15 ESP CAM

Steps

- Connect the ESP32-CAM to a computer using a USB cable for initial programming.
- Install the necessary ESP32 board definitions in the Arduino IDE (or preferred development environment).
- Select the appropriate ESP32-CAM board configuration in the IDE.
- Write code to initialize the camera, capture images, and handle network communication (Wi-Fi).
- Upload the code to the ESP32-CAM.
- Once programmed, the ESP32-CAM can be powered independently (within its voltage range of 2.2V~3.6V).
- Position the ESP32-CAM strategically to monitor parking areas or specific parking spots.

ESP32-CAM working algorithm (for parking applications):

START

1. Initialize:

- Initialize the ESP32-CAM module, including the camera sensor and Wi-Fi connection.

- Connect to the local Wi-Fi network.
- Optionally, establish a connection to a cloud platform (like Firebase as shown in Figure 4.1) for data transmission.

2. Image Capture Trigger:

- The ESP32-CAM can be triggered to capture an image based on various events:
- **Periodic Capture:** Capture images at regular intervals.
- **Motion Detection:** Integrate with a motion sensor (either external or using onboard capabilities if available) to trigger capture upon detecting movement.
- **Remote Command:** Receive a command over the network to capture an image.

3. Capture Image:

Upon receiving a trigger, the ESP32-CAM captures an image using its integrated camera.

4. Image Processing (Optional):

- The ESP32-CAM can perform basic image processing tasks onboard (depending on processing power and code implementation).

5. Transmit Image Data:

- The captured image data is transmitted wirelessly over Wi-Fi. This could involve:
- Sending the image to a cloud server for storage and analysis (e.g., to the "Model AI" for parking spot occupancy detection or license plate recognition).
- Sending processed information (e.g., detection results) to a central control system.

6. Receive and Act on Feedback (if applicable):

- The ESP32-CAM might receive feedback or commands based on the analysis of the captured images (e.g., instructions to adjust camera angle via a connected servo motor, or confirmation of parking spot occupancy).

7. Loop:

- Continuously monitor triggers and repeat the image capture and transmission process.

END

4.6 Hardware Integration

4.6.1 Liquid-crystal display

LCD display can be a useful part of your solution since it can show value from a sensor directly and able to show other messages. An LCD sensor is a great way to Display the data to Smart Door lock projects. By the Following steps, the sensor will be wired and ready to work as shown in Figure below.

Components:

- LCD (16*2).
- ESP32.
- Jumper wires.

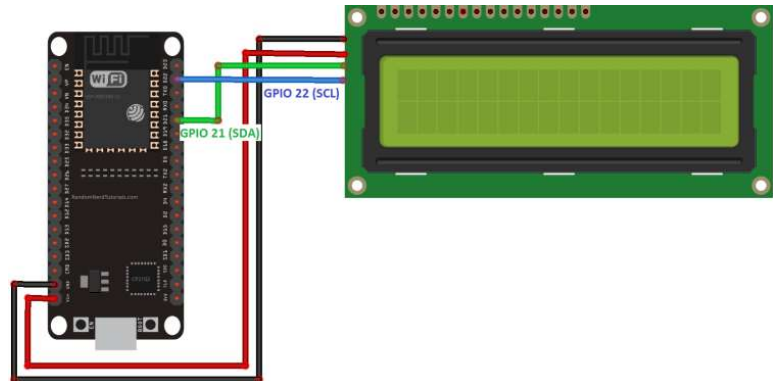


Figure 3-16 LCD With ESP

Steps

To wire your LCD screen to your board, connect the following pins:

- LCD VSS pin to GND
- LCD VCC pin to 5V
- LCD SDA pin to digital pin 21
- LCD SCL pin to digital pin 22

LCD working algorithm

Start

1. Initialize:

- Define LCD connection pins (RS, EN, D4-D7) connected to ESP32.
- Initialize the LCD using the appropriate library (e.g., Liquid Crystal) with pin configurations and display size.
- Set backlight and clear the display.

2. Display Initial Message:

- Print a starting message or instructions on the LCD.

3. Read Inputs:

- You might read data from keypad
- Store the read data in variables.

4. Process Data:

- Based on the read data or user input, perform necessary calculations or decisions.

5. Prepare LCD Output:

- Format the data or message you want to display on the LCD.
- This might involve string manipulation or converting numerical values to text.

6. Display on LCD:

- Use LCD library functions like setCursor (), print(), or println() to display the prepared message on the LCD.

7. Delay:

- Include a short delay (if needed) to allow the user to read the information on the LCD.

8. Repeat:

- Loop back to Step 3 to continuously read inputs, process data, and update the LCD display.

End

4.6.2 RFID Read/Write Module PN532 (13.56Mhz)

RFID (Radio-Frequency Identification) is a wireless communication Protocol technology that uses radio-frequency electromagnetic fields to transfer data from a tag attached to an object, for the purposes of automatic identification and tracking. You can use an RFID system to open a door. For example, only the person with the right information on his card is allowed to enter.

Parts and components of RDID:

- NFC RFID Module.
- RFID Card (13.5 MHz).
- RFID Tag (13.5 MHz).

Components:

- RFID.
- ESP32
- Jumper wires.

Steps:

- RFID VSS pin to GND.
- RFID VCC pin to 3V.
- RFID RST pin to digital pin 0.
- RFID MISO / SCL / Tx pin to digital pin 19.
- RFID MOSI pin to digital pin 23.
- RFID SCK pin to digital pin 18.
- RFID SS / SDA / Rx pin to digital pin 5.

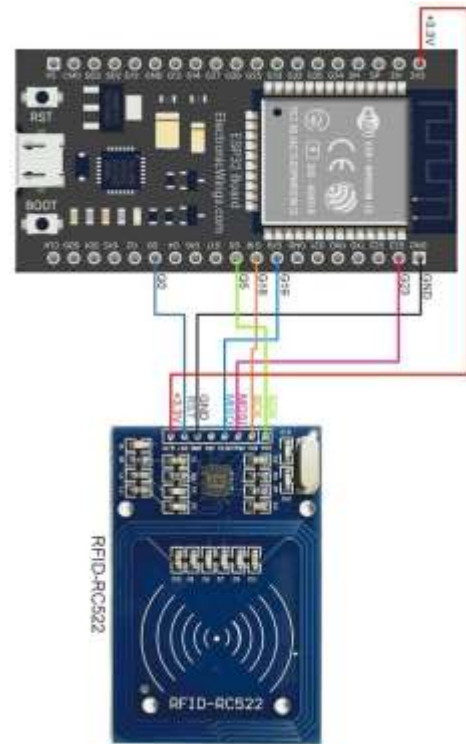


Figure 3-5 RFID With ESP32

RFID working algorithm

START

1. Initialize:

- Define RFID reader connection pins (e.g., SPI pins or specific pins depending on the reader model) connected to ESP32.
- Initialize the RFID reader library with appropriate configurations for your reader model.
- Set up power and antenna connection to the reader (refer to reader's datasheet).

2. (Optional) Display Initial Message:

- Print a starting message or instructions on the LCD (if connected) indicating readiness for RFID tag reading.

3. Read RFID Tag:

- Use the RFID library function (e.g., `read()` or specific function based on your library) to attempt reading a tag within the reader's range.
- If no tag is detected, loop back to Step 3.

4. Tag Detected:

- Identify the tag's unique identifier data read from the tag.

5. Process Tag Data:

- Based on the extracted tag ID, perform necessary actions in your program. This might involve:
 - Checking the ID against a database or list of authorized tags.
 - Triggering actions like controlling LEDs, unlocking doors, displaying messages, etc.
- You might need conditional statements to differentiate between different tag IDs.

6. Display on LCD (if connected):

- Show the detected tag ID or a success message on the LCD.

7. Delay:

- Include a short delay (if needed) to allow the user to react to the tag reading or for the system to perform additional actions.

8. Repeat:

- Loop back to Step 3 to continuously attempt reading RFID tags.

End

4.6.3 Buzzer

The piezo, also known as the buzzer, is a component that is used for generating sound. It is a digital component that can be connected to digital outputs and emits a tone when the output is HIGH. Alternatively, it can be connected to an analog pulse-width modulation output to generate various tones and

effects. The Grove Buzzer operates at both 3.3V and 5V with a sound output of 85 decibels. This module can be used to provide sound feedback to your application just like the click sound of a button on a digital watch. Are you looking to add sound to your project? The Buzzer is a great way to lock doors in a smart parking project. Through the following steps, the sensor will be connected. It is ready to operate as shown in Figure below.

Component

- Buzzer.
- ESP32.
- Jumper wires.

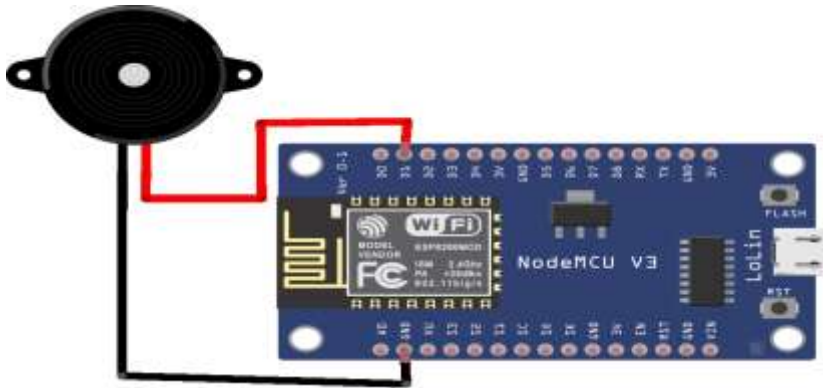


Figure 3-17 Buzzer with esp32

Steps

- Connect the Supply wire (RED) of the buzzer to the Digital Pin 3 of the ESP32 through a 100-ohm resistor.
- Connect the Ground wire (BLACK) of the buzzer to any Ground Pin on the ESP32.

Buzzer working algorithm:

START

1. Initialize:

- Define the buzzer connection pin connected to ESP32 (digital pin).
- Optionally, define a variable to store the desired frequency for the buzzer tone (in Hertz).

2. Display Initial Message (if using LCD):

- Print a starting message on the LCD (if connected) indicating the system is ready.

3. Loop:

- This is a continuous loop that can be used to generate different buzzer sounds or integrate buzzer sounds with other functionalities.

4. Generate Buzzer Sound:

- Depending on your desired sound:
 - **Continuous Tone:** Use the `tone()` function with the buzzer pin and desired frequency (if defined in step 1) as arguments.
 - **Short Beep:** Use the `tone()` function with the buzzer pin and desired frequency for a short duration (e.g., 100 milliseconds) followed by `noTone()` to stop the sound.
 - **No Sound:** If no sound is needed at this point in the loop, skip this step.

End

4.6.4 IR Sensor

The IR sensor is a component that is used for detecting infrared radiation, often employed for proximity sensing or detecting motion. It is a digital component that typically provides a digital output (HIGH or LOW) indicating the presence or absence of an object within its detection range. The Grove IR Sensor operates at both 3.3V and 5V. This module can be used to provide detection feedback to your application, just like sensing if car is approaching a gate in a smart parking project. The IR sensor is a great way to detect presence in a smart parking system. Through the following steps, the sensor will be connected. It is ready to operate as shown in Figure.

Component

- IR sensor.
- ESP32.
- Jumper wires.

Steps

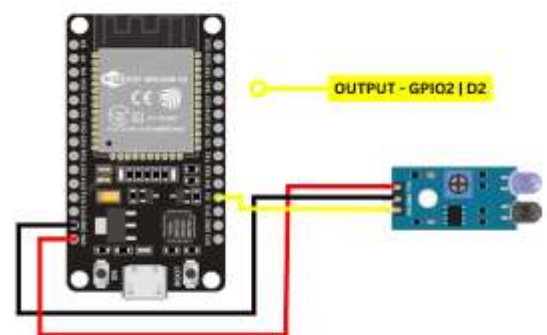


Figure 3-18 IR Sensor With ESP

- Connect the Signal wire (typically a color other than RED or BLACK) of the IR sensor to the Digital Pin 3 of the ESP32.
- Connect the Power wire (RED) of the IR sensor to a 3.3V or 5V Pin on the ESP32.
- Connect the Ground wire (BLACK) of the IR sensor to any Ground Pin on the ESP32.

IR sensor working algorithm

START

1. Initialize:

- Define the IR sensor connection pin connected to ESP32 (digital pin).
- Optionally, define a threshold value if the sensor provides an analog output (though this description suggests a digital sensor).

2. Display Initial Message (if using LCD):

- Print a starting message on the LCD (if connected) indicating the system is ready.

3. Loop:

- This is a continuous loop that monitors the state of the IR sensor.

4. Read IR Sensor State:

- Read the digital value from the IR sensor pin using `digitalRead ()`.

5. Detect Presence:

- If the digital value read from the IR sensor is HIGH (or LOW, depending on the sensor's configuration), it indicates the detection of an object.

6. Take Action (e.g., indicate parking spot occupancy):

- Based on the detected presence, it triggers a corresponding action in your smart parking system (e.g., update a display showing the spot as occupied).

7. No Detection:

- If the digital value does not indicate presence, continue monitoring the sensor in the loop.

END

4.6.5 Flame Sensor

The flame sensor, also known as the fire alarm, is a component that is used for detecting the presence of fire or flames. It is typically an analog component that outputs a voltage proportional to the intensity of the detected flame. This output can be read by a microcontroller like the ESP32 to trigger an alarm or other actions. The flame sensor operates by detecting the infrared (IR) radiation emitted by flames. This module can be used to provide fire detection capabilities to your application, just like a smoke detector in a building. The Flame Sensor is a crucial component for triggering an alert in a smart fire safety project. Through the following steps, the sensor will be connected to the buzzer and ESP32. It is ready to operate as part of the system as shown in Figure.

Component

- Flame Sensor.
- Buzzer.
- ESP32.
- Jumper wires.

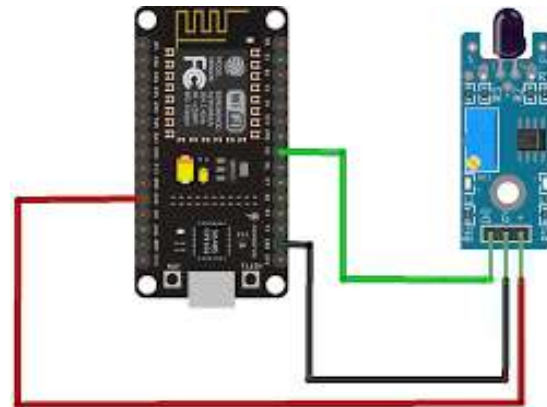


Figure 3-19 Flame Sensor With ESP

Steps

- Connect the Signal wire of the flame sensor to an Analog Input Pin (e.g., Analog Pin 34) of the ESP32.
- Connect the VCC wire of the flame sensor to a 3.3V or 5V Power Pin on the ESP32.
- Connect the Ground wire of the flame sensor to any Ground Pin on the ESP32.

- Connect the positive (+) pin of the **buzzer** to a Digital Output Pin (e.g., Digital Pin 13) of the ESP32 through a current-limiting resistor (e.g., 100-ohm).
- Connect the negative (-) pin of the **buzzer** to any Ground Pin on the ESP32.

Flame Sensor and Buzzer Working Algorithm:

START

1. Initialize:

- Define the flame sensor connection pin connected to ESP32 (analog input pin).
- Define the **buzzer** connection pin connected to ESP32 (digital output pin).
- Define a threshold value for the flame sensor reading (above which a flame is considered detected).

2. Display Initial Message (if using LCD):

- Print a starting message on the LCD (if connected) indicating the fire detection system is ready.

3. Loop:

- This is a continuous loop to monitor the flame sensor and trigger the **buzzer** if a flame is detected.

4. Read Flame Sensor Value:

- Read the analog value from the flame sensor pin using the `analogRead()` function.

5. Check for Flame:

- Compare the read sensor value with the defined threshold.

6. Trigger Buzzer (if flame detected):

- If the sensor value is above the threshold (indicating a flame):
- Set the **buzzer** pin to HIGH using `digitalWrite()` to activate the **buzzer** and sound an alarm.

- Optionally, send a notification over the internet (if the ESP32 is connected).

7. Silence Buzzer (if no flame detected):

- If the sensor value is below the threshold (no flame detected):
- Set the **buzzer** pin to LOW using digitalWrite () to turn off the **buzzer**.

8. Delay (optional):

- Introduce a small delay using delay () to control the reading frequency.

END

4.6.6 Servo motor

The servo motor in the gates of the smart parking system is an electromechanical actuator that precisely controls the opening and closing of parking barriers. It receives digital signals from the ESP32 microcontroller, which dictate the desired angular position of the motor's output shaft. This allows for smooth and controlled movement of the parking gates, regulating vehicle access to the parking facility in 6th of October City, Giza Governorate, Egypt. The servo motor replaces manual gate operation, providing an automated and efficient way to manage vehicle flow within the parking area. Through the following steps, the servo motor will be connected. It is ready to operate as shown in Figure.

Component

- Servo Motor.
- ESP32.
- Jumper wires.
- Potentially mounting hardware for the gat...

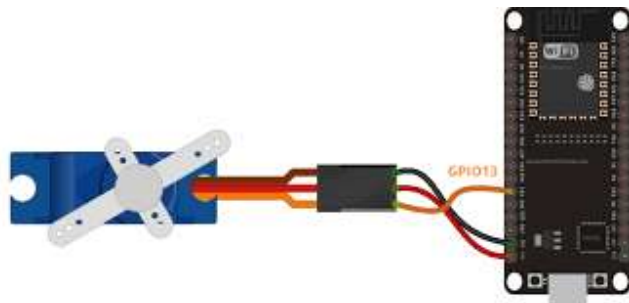


Figure 3-20 Servo Motor With ESP

Steps

- Connect the Power wire (typically RED or Orange) of the servo motor to a 5V power source on the ESP32 or a separate 5V power supply.

- Connect the Ground wire (typically BLACK or Brown) of the servo motor to a Ground Pin on the ESP32 or the ground of the separate power supply.
- Connect the Signal wire (typically YELLOW, White, or Orange) of the servo motor to a Digital Pin on the ESP32 (for example, Digital Pin 3).

Servo Motor working algorithm

START

1. Initialize:

- Define the servo motor's signal pin connected to the ESP32 (digital pin).
- Include the Servo library in your ESP32 code.
- Create a Servo object to control the motor.
- Attach the Servo object to the defined signal pin using the attach () function.
- Optionally, set an initial position for the parking gate (e.g., fully closed).

2. Receive Control Signal:

- The ESP32 receives a signal (e.g., from a sensor indicating a vehicle approaching, a signal from an RFID reader, or a command from a central management system).

3. Determine Gate State:

- Based on the received signal and the system's logic, determine the desired state of the parking gate (open or closed).

4. Control Servo Position:

- **To Open:** Use the write () function with the Servo object and the angle corresponding to the fully open position of the gate (typically around 0 or 180 degrees, depending on the servo and gate mechanism).
- **To Close:** Use the write () function with the Servo object and the angle corresponding to the fully closed position of the gate.
- You might introduce a small delay after each write () command to allow the servo to reach the desired position.

5. Loop:

- Continuously monitor for new control signals to adjust the gate position as needed.

END

4.7 System Design

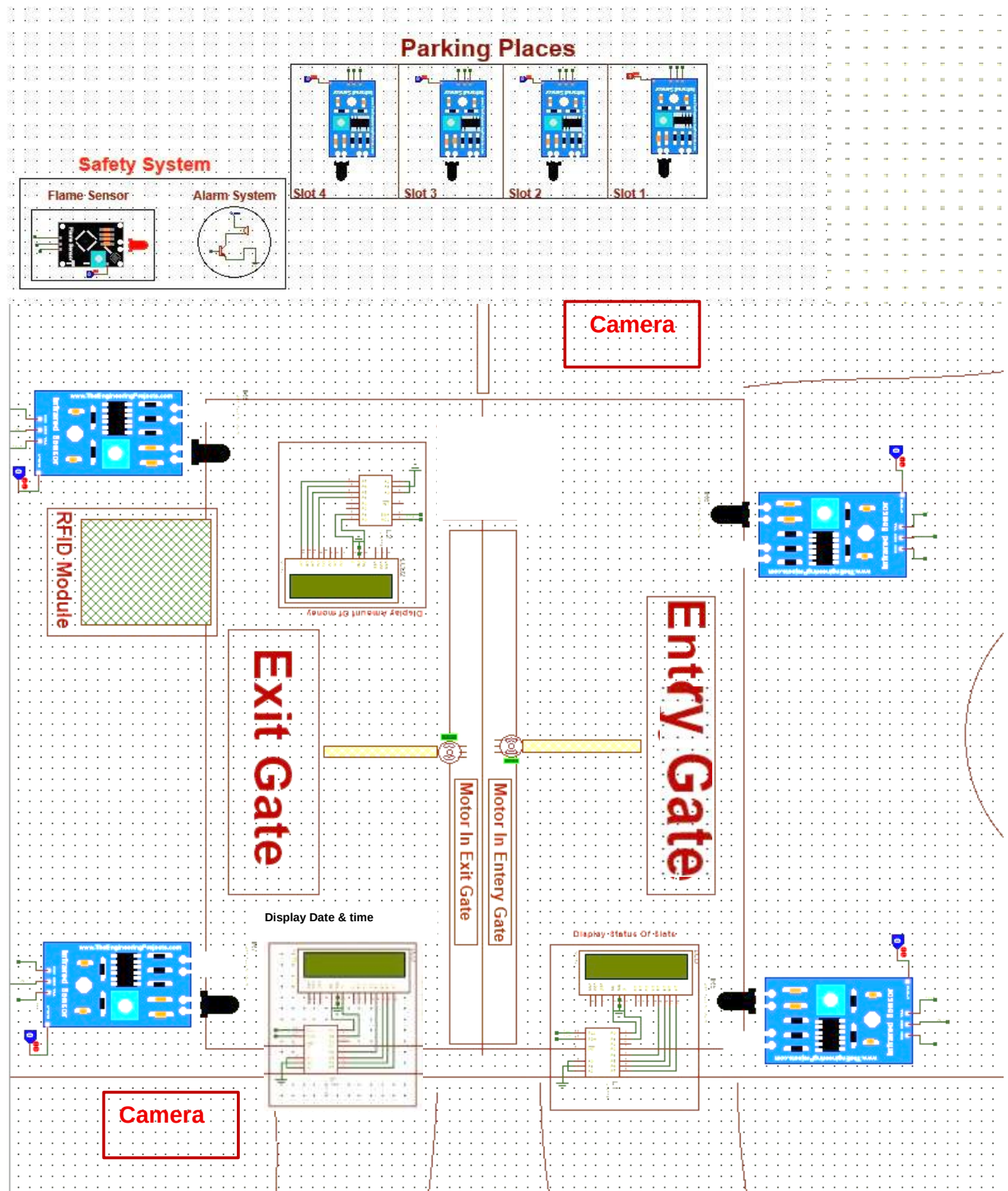
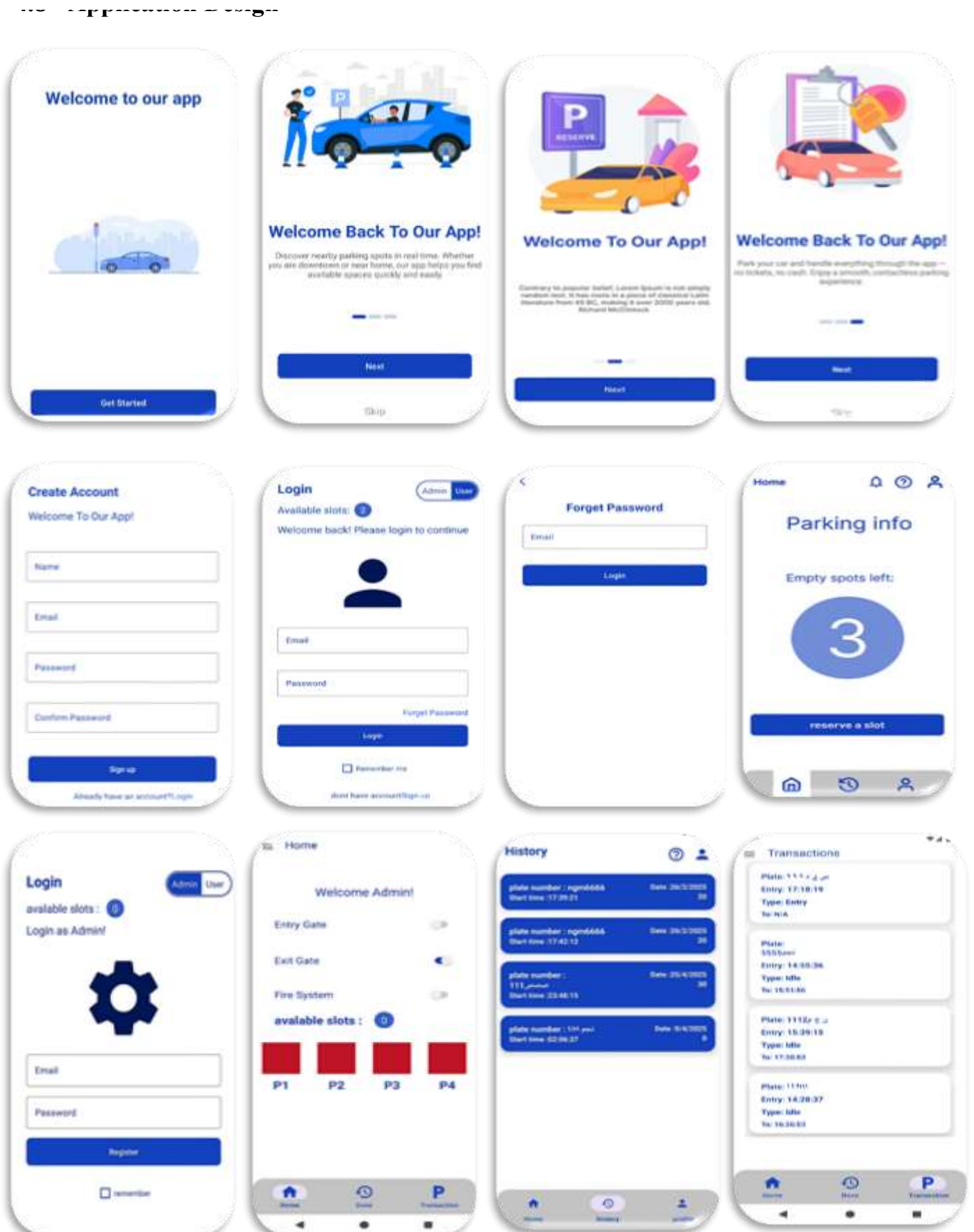


Figure 3-21 System Design



4.9 System Scenario

Figure 3-22 Application Design

Case one:

1. User Reserved for application a specific time
2. When users arrive at the parking space in the specific entry gate the cam will take shot for the plate number of the car and make some filters using image processing to enhance the image and after that give image to model ai (Yolov11 and OCR) to detect plate license and extract text in the plate.
3. After the model gives me a response after checking in the database if this plate in reserved form app the entry gate will be open automatically.
4. When arrive the exit gate cam will take a shot and extract license of the plate and check the car in which time entry the parking and any time exit the parking and calculate different if it take larger than that reserved from app will be payment the difference by master card else user parking in the specific time that reserved from app the gate will be open automatically.

Case two:

1. If the customer **does not have** an application and needs to park your own car the user will go to the car park.
2. When you arrive at the entry gate ESP32 Cam take shot and extract data form license of the car and the system then check if this car is reserved from the application or not the system finds after searching in the database that this car not reserved form application.
3. then the system check if there are and empty places in the parking or not if the parking contain empty places the entry gate will open automatically and the system will calculate the time form the entry gate opened until this car need to go out parking and display on the **lcd the total amount** of money and time you spend in the parking and the user will be payment using master card to go out the parking by using module RFID

Case three:

1. Parking contains sensor for detecting fire in the parking for safety. If this sensor detects the fire the two gated entry and exit will open automatic, and the alarm sound will be turned on for alert customers to take care

4.10 Software Tools Used

4.10.1 Arduino IDE:

The Arduino IDE (Integrated Development Environment) is a software platform specifically designed for programming Arduino boards. It provides an intuitive interface and a set of tools that enable users to write, compile, and upload code to Arduino microcontrollers.



Figure 3-23 Arduino IDE

Arduino IDE Features

- **User-friendly interface:** The Arduino IDE offers a simple and easy-to-use interface, making it accessible even for beginners in programming and electronics.
- **Cross-platform compatibility:** It is available for multiple operating systems such as Windows, macOS, and Linux, allowing users to work with their preferred platform.
- **Code editor:** The IDE includes a built-in code editor with features like syntax highlighting, auto-indentation, and code completion, facilitating the writing and editing of Arduino code.
- **Library management:** It provides a library manager that allows users to easily search, install, and update libraries, which are pre-written code modules that extend the functionality of Arduino boards.
- **Compilation and uploading:** The IDE seamlessly handles the compilation process, translating the written code into machine-readable instructions for the Arduino board. It also enables users to upload the compiled code to the board for execution.
- **Serial monitor:** Arduino IDE includes a serial monitor tool that allows users to interact with their Arduino board through a computer's serial port, facilitating communication and debugging.

4.10.2 Proteus Design Suite program

The Proteus Design Suite is a proprietary software tool suited to use primarily for electronic design automation. The software is used mainly by electronic design engineers and technicians to create schematics and electronic prints for manufacturing printed circuit boards.



Figure 3-24 Proteus Design

Proteus Design Suite Features

1. It was developed in Yorkshire, England by Lab center Electronics Ltd and is available in English, French, Spanish and Chinese languages.
2. It is integrated with the simulation of real time simulation of electronic circuits and testing whether the designed circuits are working properly or not.

4.10.3 Draw.io

is a web-based diagramming tool that allows users to create and collaborate on a wide range of diagrams, flowcharts, and visual representations. It offers a user-friendly interface with drag-and-drop functionality, making it easy to create professional-looking diagrams without the need for advanced design skills.



Figure 3-25 Draw.io

4.10.4 Microsoft Word:

Microsoft Word is the most popular word processing program – and justifiably. It is easy to use and allows you to create all different types of documents.



Figure 3-26 Office Applications

4.10.5 Visual Studio Code

Visual Studio Code is a free, lightweight yet powerful source code editor loved by many. It offers intelligent code completion, debugging tools, and Git integration for efficient coding. Its vast extension marketplace allows customization for various languages and workflows. Cross-platform and fast, VS Code is a top choice for developers.



Figure 3-27 Visual Studio Code

4.10.6 Android Studio

Android Studio is the official IDE for developing Android applications, based on IntelliJ IDEA. It provides a dedicated environment for coding in Java, Kotlin, and C++. Key features include visual layout editor, debugging and profiling tools, and Gradle-based build automation. It offers extensive support for the Android SDK and integrates with Google Cloud Platform. With its rich features and constant updates, it's the standard for Android development.



Figure 3-28 Android Studio

a

Chapter 5

System Implementation

5 Chapter 5

System Implementation

5.1 Introduction

System implementation marks a crucial phase in the life cycle of any technological solution, involving the transition from design and development to the practical application of the system in a real-world environment. This phase encompasses a series of carefully planned and executed steps to deploy, configure, and integrate the new system into the existing infrastructure.

In this phase, the chapter is separated into three parts:

- **Mobile Application Implementation:**

This section details the development and implementation of the mobile application. It covers aspects such as user interface design, features, functionalities, and the technologies used for development.

- **Hardware Implementation:**

This part focuses on the hardware components of the project. It includes the description of hardware components used, specifications, connections, and setup.

- **Integration with Firebase:**

This section explains the process of connecting the mobile application with the hardware using Firebase. It covers setting up Firebase, establishing communication protocols, and ensuring data exchange between the mobile application and the hardware components.

5.1.1 Mobile App Implementation

Our smart parking system leverages the power of native Android development to deliver robust and efficient mobile experience. The application was built using Android SDK and Kotlin, taking full advantage of platform-specific features and performance optimizations. Android Studio served as our primary Integrated Development Environment (IDE), providing comprehensive tools for development, debugging, and testing. We utilized Android Jetpack libraries, such as Compose for a modern UI and Room Persistence Library for local data management, to ensure a high-quality and maintainable application. Android Studio's

emulator and device debugging capabilities were crucial for iterative development and thorough testing across various Android versions and devices. While cross-platform solutions exist, we chose native Android development to maximize performance, provide a seamless user experience, and have direct access to the full spectrum of Android APIs for features like location services and background processing essential for a smart parking system.

5.1.2 Android (Native) Have Multiple Advantages

1. Performance and Speed

- **Optimized for the Platform:** Native apps are built using the core programming languages and APIs of the Android platform. This allows them to run faster and more efficiently because the code is directly executed by the device's operating system without any intermediary layers.
- **Direct Access to Hardware:** Native apps have immediate and complete access to all of the device's hardware capabilities, such as the camera, GPS, sensors (accelerometer, gyroscope), Bluetooth, and more. This enables developers to create rich and highly interactive.
- **Responsiveness:** Native apps tend to be more responsive and provide smoother user experience, especially with complex graphics, animations, and transitions.

2. User Experience (UX) and User Interface (UI)

- **Native Look and Feel:** Native apps adhere to the specific UI/UX guidelines and design principles of the Android operating system (e.g., Material Design). This results in a familiar and intuitive user experience that users are accustomed to, leading to higher user satisfaction.
- **Seamless Integration:** Native apps seamlessly integrate with other native features and applications on the device, such as sharing functionalities, notifications, and system settings.

3. Security

- **Leveraging OS Security Features:** Native apps can take full advantage of the built-in security features provided by the Android operating system, offering better protection against vulnerabilities and threats.

- **Trusted APIs:** Native development primarily relies on official and well-tested APIs provided by Google, reducing the risk associated with third-party libraries that might have security flaws.

4. Access to the Latest Features

- **Immediate Access to New SDK Features:** Native developers get access to the newest features and APIs in the Android SDK as soon as they are released by Google. This allows them to incorporate cutting-edge functionalities into their apps without delay.

5. Stability and Reliability

- **Platform-Specific Optimization:** Because native apps are developed for a single platform, they can be thoroughly tested and optimized for that specific environment, generally leading to more stable and reliable applications.
- **Better Bug Detection:** Developing a single platform with its dedicated tools and SDK can sometimes make it easier to identify and fix bugs compared to cross-platform development where issues might arise from the interaction between different layers.

6. Scalability

- **Easier to Scale with Platform Advancements:** As the Android platform evolves, native apps can be more easily updated and scaled to take advantage of new hardware and software capabilities.

7. Development Tools and Ecosystem

- **Robust IDE (Android Studio):** Android Studio, the official IDE for Android development, provides a comprehensive set of tools for coding, debugging, profiling, and testing native Android applications.
- **Extensive Libraries and Frameworks:** The Android SDK offers a wide range of well-documented libraries and frameworks that simplify the development process for various functionalities.
- **Large and Active Community:** The Android developer community is vast and active, providing ample resources, support, and third-party libraries.

5.2 Hardware implementation

We Used Arduino IDE and ATMEL studio in our parking Automation System for implementing the hardware for all the sensors and components.

By using the Arduino IDE for hardware implementation, we were able to take advantage of its affordability, cross-platform compatibility, and ease of programming. The rich ecosystem of libraries and examples provided by the Arduino community made it easy to add functionality to our project, while the user-friendly interface of the Arduino IDE made the development process smooth and efficient. Overall, Arduino IDE proved to be an excellent choice for implementing the hardware component of our project.

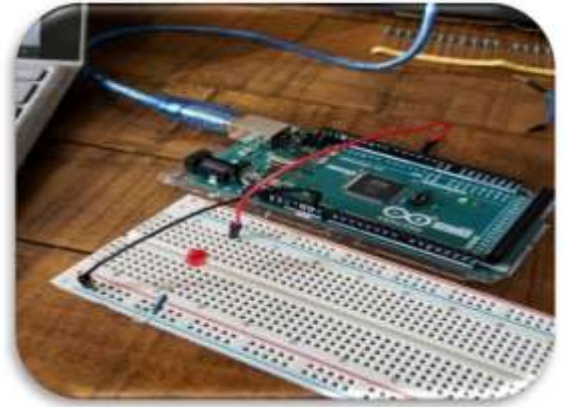


Figure 5-1 Hardware Setup

Firestore Integration

Firestore is a comprehensive mobile development platform provided by Google. It offers a wide range of features and services to help developers build high-quality apps, grow their user base.

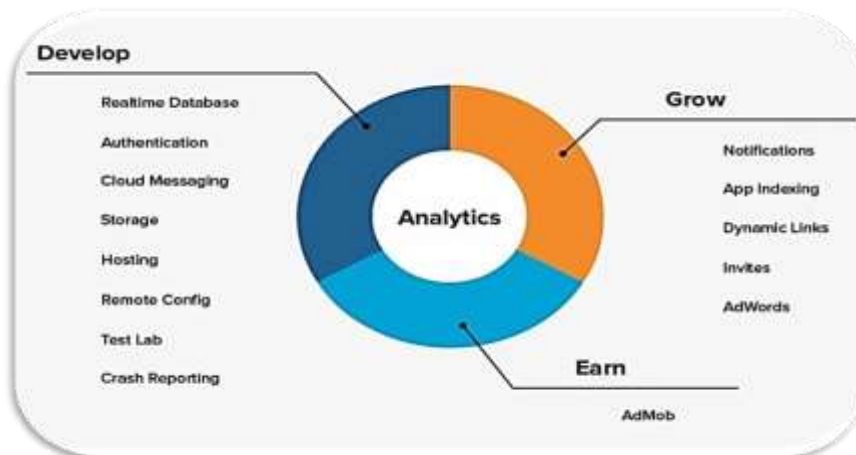


Figure 5-2 Firestore Integration

5.3 Features Implemented Using Firebase

1. Firebase Authentication

We used Firebase Authentication to handle user authentication in our mobile application. Firebase Authentication supports various authentication methods, including email/password, phone number, Google sign-in, Facebook login, and more. By integrating Firebase Authentication, we ensured that user data is secure and that only authenticated users can access the app's features.

2. Real-time Database (Fire store)

We implemented Fire store, Firebase's real-time database, to store and sync data between the mobile application and the backend server. Fire store provides Realtime synchronization, offline support, and automatic scaling, making it ideal for building real-time collaborative applications. With Fire store, data changes are synchronized in real-time across all clients, ensuring that users always have the latest information.

3. Firebase Cloud Messaging (FCM)

We integrated Firebase Cloud Messaging (FCM) to send push notifications to users and keep them engaged with the application allowing targeted messaging, topic-based messaging, and custom notification content, enabling personalized and timely communication with users. With Firebase Cloud Messaging, we were able to re-engage users, promote new features, and provide important updates to our users in

5.3.1 Implementation Process

1. Setting up Firebase

We created a Firebase project in the Firebase console and added our mobile application to the project. We configured Firebase Authentication to enable the desired authentication methods for our application.

2. Integrating Firebase SDK

We added the Firebase SDK to our Flutter project by including the necessary dependencies in the pubspec.yaml file. We initialized Firebase in our application by adding the Firebase configuration files provided by the Firebase console.

3. Implementing Firebase Authentication

We implemented user authentication using Firebase Authentication by adding sign-up, sign-in, and sign-out functionality to our application. We customized the authentication flow to match the design and user experience requirements of our application.

4. Using Fire store for Data Storage

We used Fire store to store and sync user data, application settings, and other relevant information between the mobile application and the backend implemented CRUD (Create, Read, Update, Delete) operations to interact with Fire store and manage application data.

5.3.2 Advantages of Firebase Implementation

1. Scalability and Performance:

Firebase is built on Google's infrastructure, providing automatic scaling and high performance, even as our user base grows.

2. Real-time Updates

With Firebase Realtime Database and Fire store, data changes are synchronized in real-time across all clients, ensuring that users always have the latest information.

3. Security

Firebase Authentication offers robust security features, including password hashing, account linking, and access control, to help protect user data and prevent unauthorized access.

4. Analytics and Insights

Firebase Analytics provides valuable insights into user behavior, app usage, and engagement, helping us make data-driven decisions to improve our application.

Chapter 6

Communication

Protocols

6 Chapter 6

Communication Protocols

6.1 Introduction

Communication protocols, such as UART, I2C, and SPI, are sets of rules governing data exchange. These specific protocols ensure different types of devices can understand each other for effective communication. They define message formats, transmission methods tailored to their purpose, and error handling mechanisms. These protocols are essential for enabling seamless data transfer across various connections. Utilizing standardized protocols like UART, I2C, and SPI allows for interoperability between a wide range of diverse systems and components.

6.2 I2C Overview

I2C is a two-wire serial communication protocol using a serial data line (SDA) and a serial clock line (SCL). The protocol supports multiple target devices on a communication bus and can also support multiple controllers that send and receive commands and data. Communication is sent in byte packets with a unique address for each target device

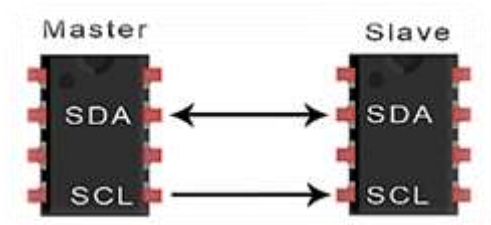


Figure 6-1 I2C Protocol

- **Inter-Integrated Circuit (I2C):** I2C is a serial communication protocol that allows multiple devices to communicate with each other on a single bus. It's widely used in embedded systems due to its simplicity and low cost.
- **Two-Wire Interface:** I2C uses only two wires:
 - **SDA (Serial Data):** Carries the data bits.
 - **SCL (Serial Clock):** Synchronizes the data transfer.

6.2.1.1 Key Features of I2C

- **Multi-Master/Multi-Slave:** Multiple devices can act as masters (initiating communication) or slaves (responding to requests).

- **Simple Hardware:** Requires only two wires, making it easy to implement.
- **Low Speed:** Typically operates at speeds up to 100 kbps, although some devices can support higher speeds.
- **Flexible Addressing:** Each device on the bus has a unique 7-bit address.
- **Clock Stretching:** Devices can temporarily halt the clock to allow for slower data transfers or processing.

6.2.2 How I2C Works

- 1) **Start Condition:** The master device initiates communication by pulling the SDA line low while the SCL line is high.
- 2) **Address Phase:** The master sends a 7-bit address followed by a read/write bit (0 for write, 1 for read).
- 3) **Data Transfer:**
 - **Write:** The master sends data bytes to the slave.
 - **Read:** The slave sends data bytes to the master.
- 4) **Stop Condition:** The master terminates communication by pulling the SDA line high while the SCL line is high.

6.2.3 Frame Format

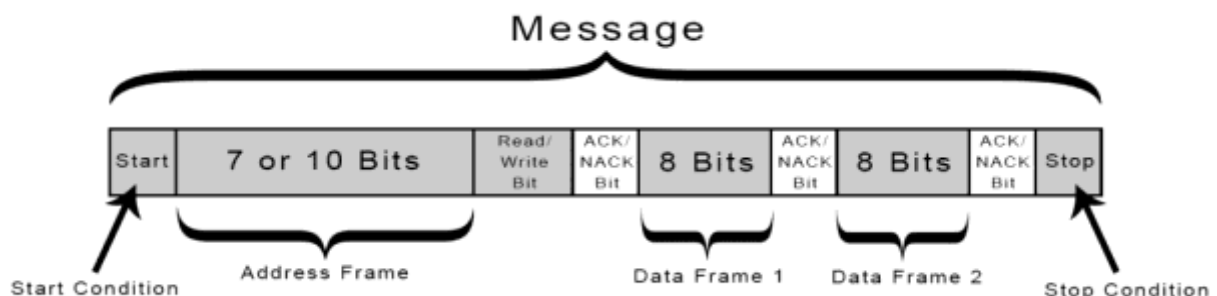


Figure 6-2 I2C Frame Format

6.3

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used for short-distance communication, primarily in embedded systems. It operates on a master-slave architecture, where a single master device controls

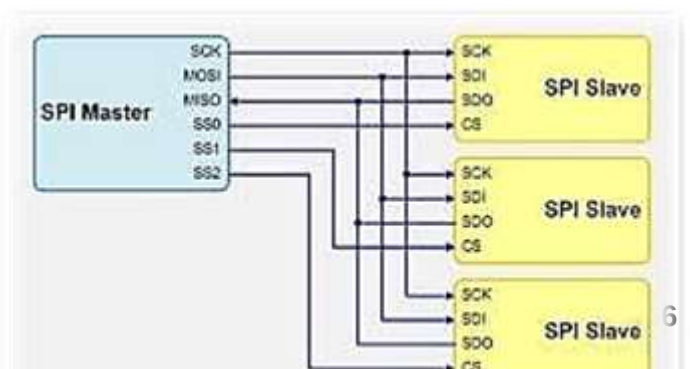


Figure 6-3 SPI Protocol

one or more slave devices. SPI uses four dedicated lines: Serial Clock (SCLK), Master Out Slave In (MOSI), Master In Slave Out (MISO), and Slave Select (SS). The master initiates communication and generates the clock signal, synchronizing data exchange. Data is transmitted and received simultaneously (full duplex) bit by bit. The master selects a specific slave by activating its SS line, allowing for communication with multiple slaves on the same bus.

6.3.1 Key features of SPI include

- **Synchronous communication:** Relies on a clock signal for synchronization.
- **Full duplex:** Allows simultaneous transmission and reception of data.
- **High-speed data transfer:** Generally faster than other serial protocols like I2C.
- **Simple protocol:** Relatively easy to implement with hardware and software.
- **Master-slave architecture:** One master control multiple slaves.
- **Flexible data length:** Data transfer is not limited to 8-bit words.

6.3.2 How SPI Work

1. Master-Slave Relationship

- The **master** device initiates and controls communication. It generates the clock signal and selects the slave device it wants to communicate with. Typically, a microcontroller acts as the master.
- **Slave** devices respond to the master's commands. They cannot initiate communication on their own. Peripherals like sensors, memory chips, and displays often act as slaves.

2. Physical Connections (4 Wires)

- **SCLK (Serial Clock):** The master generates this clock signal, which synchronizes data transmission between the master and slave.

- **MOSI (Master Out Slave In):** This line carries data from the master to the selected slave device.
- **MISO (Master In Slave Out):** This line carries data from the selected slave device back to the master.
- **SS/CS (Slave Select/Chip Select):** The master uses this line to select which slave device it wants to communicate with. This line is typically active-low (meaning the slave is selected when the line is pulled low). Each slave device usually has its own dedicated SS/CS line connected to the master.

3. Communication Process

- **Slave Selection:** To begin communication with a specific slave, the master pulls the slave's corresponding SS/CS line low. All other slaves on the bus ignore the communication while their SS/CS line remains high.
- **Clock Generation:** The master generates a clock signal on the SCLK line. The frequency of this clock determines the data transfer speed.
- **Data Transmission (MOSI):** The master sends data bit by bit to the slave on the MOSI line. The data is typically shifted out on one edge of the clock signal (rising or falling, depending on the configured SPI mode).
- **Data Reception (MISO):** Simultaneously, if the slave needs to send data back to the master, it does so bit by bit on the MISO line, synchronized with the same clock signal. This allows for full-duplex communication (simultaneous sending and receiving).
- **Data Sampling:** The master and slave devices sample (read) the data on the MOSI and MISO lines on the opposite clock edge from when the data is shifted out. This ensures that the data is stable when it is read.
- **Transaction Completion:** After the desired number of data bits have been transferred, the master stops toggling the clock signal and deselects the slave by pulling its SS/CS line high.

4. SPI Modes

- The SPI protocol offers flexibility in terms of clock polarity (CPOL) and clock phase (CPHA). These two settings define four different SPI modes (0, 1, 2, and 3) that determine the idle state of the clock, and the clock edges used for data sampling and shifting. The master and slave devices must be configured to use the same SPI mode for successful communication

6.3.3 Frame Format

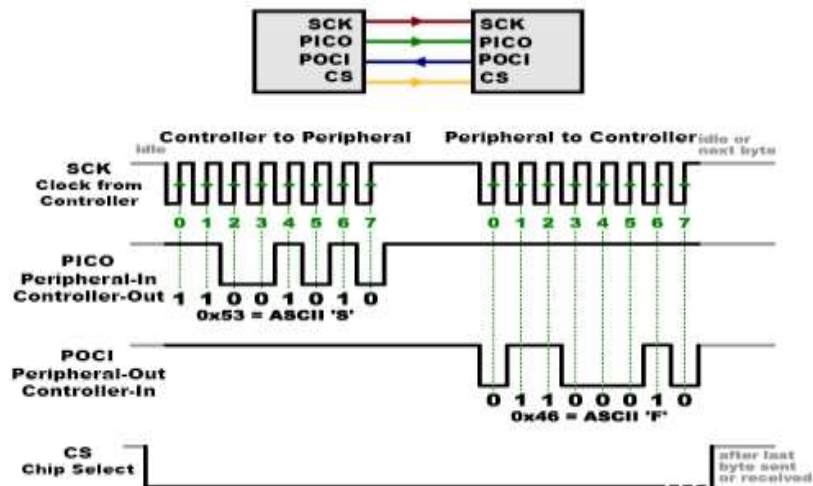


Figure 6-4 SPI Frame Format

Chapter 7

plate Recognition

with AI

7 Chapter 7

plate Recognition with AI

7.1 Introduction

License Plate Recognition (LPR), also known as Automatic Number Plate Recognition (ANPR), is a computer vision technology used to identify vehicles based on their license plates. It is a critical function in

various domains such as intelligent transportation systems, law enforcement, traffic monitoring, access control, and automated parking management.

As cities grow and traffic volumes increase, the need for automated and efficient vehicle identification systems becomes essential. Traditional methods of license plate recognition often struggle under real-world conditions including poor lighting, image noise, motion blur, and varied plate formats across regions and countries.

The rise of artificial intelligence (AI) and deep learning has transformed the landscape of LPR systems. By leveraging advanced visual recognition techniques, modern systems can now perform accurate plate detection and character recognition across a wide range of scenarios, achieving high speed and reliability in both urban and rural environments.

7.2 Related Work

Table 7-1 Related Work

References	Data Set	Algorithm	Highest Accuracy
Amany Sarhan, Shahd Hatem, Bassel Darwish, Mohamed Ramadan [1]	EALPR dataset	YOLOv8, Easy-OCR, and CNN	YOLOv8 achieved 94.2%, Easy-OCR achieved 94% and CNN achieved 93%
Tarteel Abdelfattah Elattar, Youssef Abdelrahman, Mohammed Abdel-Megeed, Shereen Moataz [2]	EMLPR, EALPR dataset	YOLOv8 and Easy-OCR	YOLOv8 achieved 100% and Easy-OCR achieved 80%
Mohammad M. Abdellatif, Noura H. Elshabasy, Ahmed E. Elashmawy, Mohamed AbdelRaheem [3]	Not mentioned	Simple OCR (no DL)	OCR achieved 76 %
Sahar Bayoumi, Ezzat Korany, Sherin Aly [4]	221 car images	ANN (no DL)	achieved 78 %

7.3 Tools and Technologies Used

To build an effective and reliable AI-based license plate recognition system, we utilized a combination of advanced object detection, optical character recognition, and real-time hardware integration. This section outlines the core tools and technologies used in our system. Technical details and training processes are discussed in later sections.

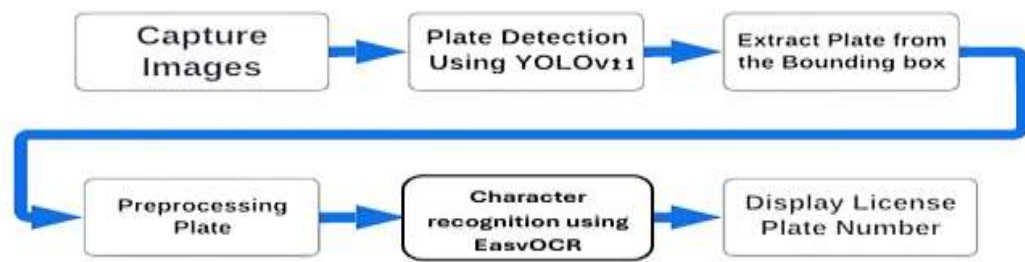


Figure 7-1 General Process

1. Object Detection Model – YOLO V11

We employed YOLO V11 (You Only Look Once, Version 11) to detect and localize vehicle license plates within images. This model offers fast and accurate detection, making it highly suitable for real-time scenarios such as smart parking systems. YOLO V11 was customized to focus specifically on the spatial features of vehicle plates.

2. Optical Character Recognition – EasyOCR

EasyOCR was used to extract the textual content from detected license plates. It supports multiple languages and is capable of recognizing characters under challenging conditions such as low lighting, distorted angles, and noisy backgrounds. The model was further adapted to handle Arabic characters in addition to English.

3. Custom Vehicle Plate Dataset

We collected and annotated a dataset tailored to our use case. It includes images of vehicles captured in various environments, lighting conditions, and angles. This dataset was used to train and evaluate both the detection and OCR components of the system.

Attribute	Value
Total Images	2087 images
Total Labels	2088 labels
Total Size	206.19 MB
Image Sources	Street photography, parking areas, traffic scenes
Classes	1 class (License Plate)
Image Conditions	Different lighting, angles, and distances
Annotations	Bounding boxes manually labeled
Data Split	80% Training, 20% Validation

Figure 7-2 plate dataset

7.4 YOLO V11 Overview

YOLO (You Only Look Once) is a real-time object detection algorithm that has gained significant popularity due to its speed and accuracy. Unlike traditional detection pipelines that separate object localization and classification into multiple stages, YOLO performs both tasks in a single neural network pass. This unified approach dramatically improves inference time, making it highly effective for time-sensitive applications such as vehicle license plate detection. Version 11 of YOLO introduces enhancements

in model architecture and feature representation, specifically aimed at improving detection accuracy in challenging scenarios—including small objects, low-light conditions, and overlapping instances.

Key Architectural Components of YOLO V11

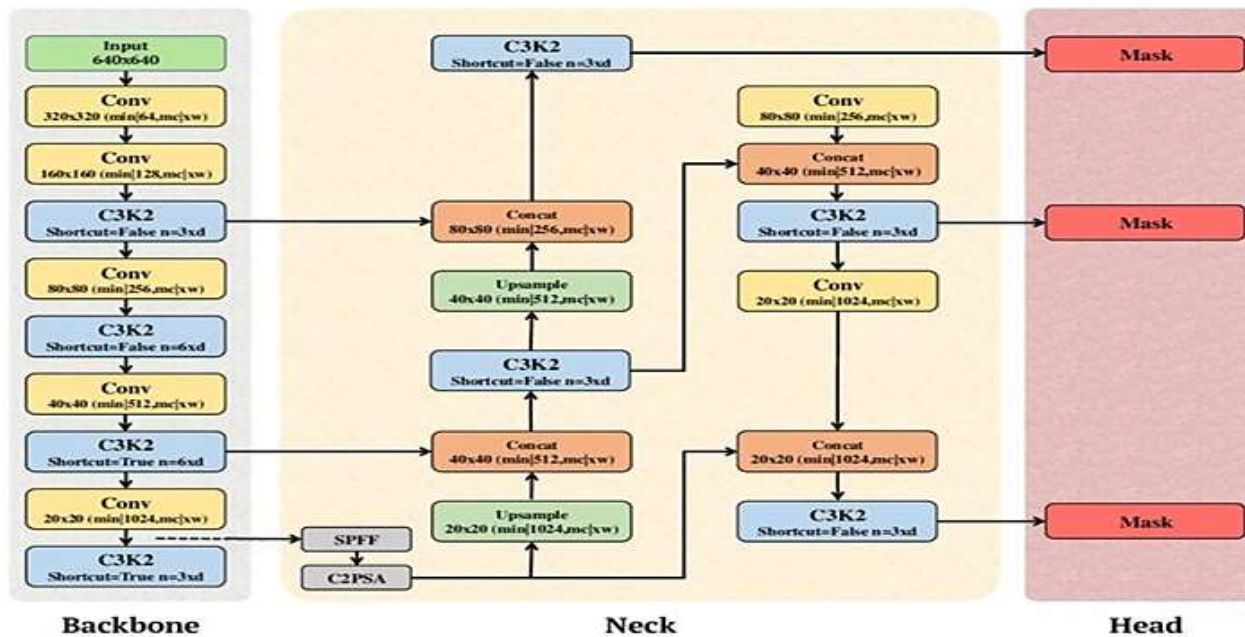


Figure 7-3 Yolo Architecture

- **Backbone:** A deep convolutional neural network responsible for extracting low-level and high-level spatial features from the input image. YOLO V11 uses an improved backbone to capture finer details in small targets such as license plates.
- **Neck:** The neck integrates Feature Pyramid Networks (FPN) and Path Aggregation Networks (PANet) to combine multi-scale features, which is crucial for detecting objects of varying sizes and orientations.
- **Detection Head:** This module generates bounding box predictions along with objectless scores and class probabilities. It is optimized to balance precision and recall, especially in real-time environments.

Why YOLO V11?

YOLO V11 was selected for this project due to its:

- High accuracy in detecting small and medium-sized objects like license plates
- Real-time processing capability, which aligns with the timing constraints of smart parking systems

- Compatibility with edge deployment using lightweight hardware accelerators (e.g., ESP32-CAM input streams)

In our implementation, YOLO V11 serves as the first stage in the plate recognition pipeline—responsible for identifying the precise region of interest (ROI) that contains the license plate. This ROI is then passed to an OCR engine for text extraction.

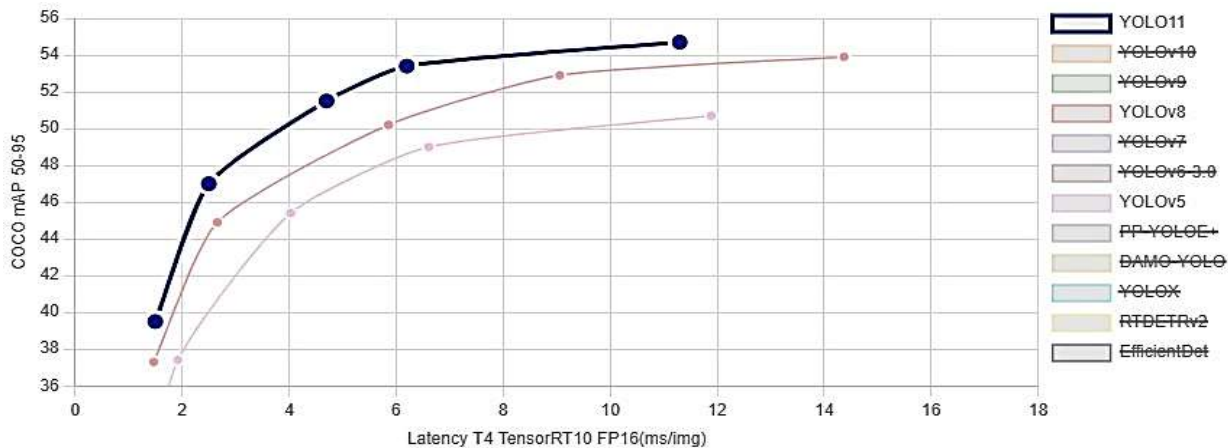


Figure 7-4 compare between different versions of yolo

Yolo Model: (You Only Look Once) is a **Supervised Learning** and **deep learning**(because it's a type of convolutional neural network (CNN)) model for object detection that divides an input image into a grid and, in a single forward pass, predicts bounding boxes and class probabilities for objects within those grid cells, making it extremely fast and efficient for real-time applications.

How yolo algorithm work?

Our system models detect a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

- The exact grid sizes depend on the architecture and input resolution.
- $S*S$ may be $(13 \times 13, 26 \times 26, 52 \times 52)$. in v11

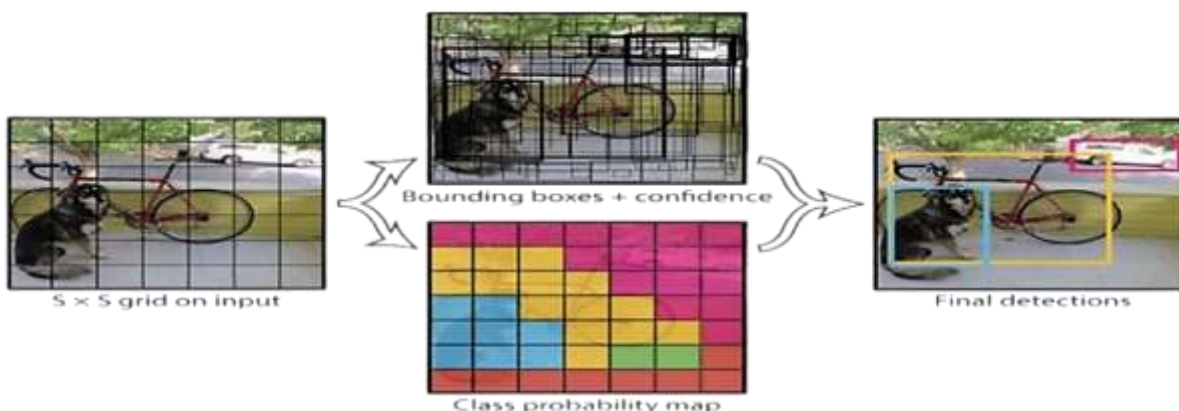


Figure 7-5 yolo Algorithm

7.5 Confusion matrix

A confusion matrix is a table that helps you understand how well a classification model works. It compares the predicted labels with the actual labels.

Why is it useful?

It helps you calculate:

- Accuracy
- Precision
- Recall

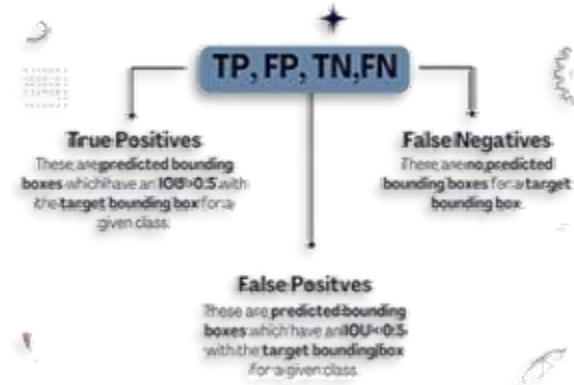


Figure 7-6 Confusion matrix explanation

7.6 YOLO V11 Fine-Tuning

The base YOLO V11 model used in our project was pretrained on the **COCO dataset** (Common Objects in Context), a large-scale benchmark in computer vision. COCO includes images labeled across 80 general object categories such as people, cars, animals, and household items—but it does not include license plates as a target class.

Table 7-2 fine tuning Yolo

Item	Value
Base Model	YOLOv11
Pre-trained on	COCO Dataset
Fine-tuned on	Egyptian License Plate Dataset
Number of Epochs	20
Optimizer	Adam (Adaptive Moment Estimation)
Learning Rate	0.0002

especially those with region-specific features such as Arabic script, plate color schemes, and layout styles commonly found in Egypt.

To address this limitation, we performed fine-tuning on a Egyptian license Plate Dataset composed entirely of vehicle images with Arabic license plates. This dataset was designed to reflect real-world challenges such as:

- Arabic-only text content with various fonts and styles

- Non-standard plate formats unique to Egyptian vehicles
- Environmental diversity, including day/night lighting, motion blur, and angled views
- Each image was manually annotated with a bounding box tightly surrounding the license plate area. These annotations enabled the model to learn the exact positioning and visual patterns associated with Egyptian plates.
- The fine-tuning process was conducted **over 20 epochs**, allowing the model to adapt effectively to the specific characteristics of the dataset while avoiding overfitting. Data augmentation techniques such as rotation, brightness shifts, and blurring were applied to increase robustness.
- Fine-tuning YOLO V11 on this domain-specific dataset significantly improved its detection performance. The model became capable of consistently identifying Arabic license plates across a wide range of practical parking scenarios, something that was not possible using the original COCO-trained weights alone.

7.7 EasyOCR Overview

EasyOCR is an open-source Optical Character Recognition (OCR) engine that leverages deep learning to extract text from natural images. It supports over 80 languages, including Arabic, making it an ideal choice for recognizing license plate text in regions where non-Latin scripts are used.

Unlike traditional OCR systems such as Tesseract, which rely heavily on handcrafted features and strict formatting, EasyOCR is built on a modern deep learning pipeline that enables it to recognize text in a wide range of real-world conditions, **including:**

- Low-resolution or blurred images
- Curved, angled, or skewed text regions
- Multilingual and mixed-script content (e.g., Arabic + numbers)
- Noisy or cluttered backgrounds

7.8 Core Architecture

EasyOCR's architecture combines several deep learning components:

- **Convolutional Neural Networks (CNNs)** are used to extract features from the input image. These features capture local textures and spatial patterns, which are critical for distinguishing individual characters.

- **Recurrent Neural Networks (RNNs)**, typically bi-directional LSTMs, are used to process the sequence of features over time, allowing the model to capture dependencies between characters.
- **Connectionist Temporal Classification (CTC)** is applied as the output layer. It enables the model to make predictions without requiring perfectly segmented characters, making it well-suited for real-world applications where characters may overlap or be partially obscured.

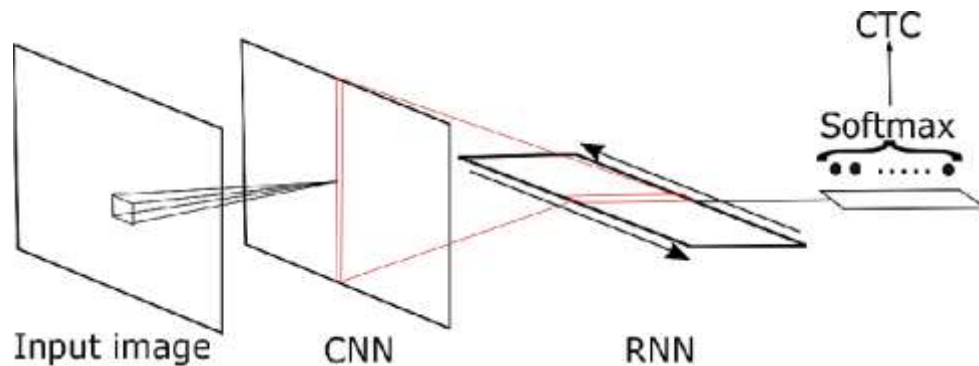


Figure 7-7 OCR Architecture

Why EasyOCR for License Plate Recognition?

EasyOCR was selected for this project due to its:

- Strong performance on multilingual and non-Latin scripts, especially Arabic
- Ability to operate on noisy or real-time images captured from embedded devices
- Lightweight design and ease of integration into Python-based AI pipelines
- Pretrained models that can be further fine-tuned for domain-specific tasks

In our system, EasyOCR is used as the second stage in the license plate recognition pipeline. Once YOLO V11 detects and localizes the license plate, the cropped region is passed to EasyOCR to extract the alphanumeric content. This two-stage approach allows for highly accurate and efficient end-to-end recognition.

7.9 EasyOCR Fine-Tuning

Although EasyOCR comes with strong pretrained models, they are designed for general-purpose OCR tasks and may not perform optimally on domain-specific inputs such as **Arabic license plates**, especially when dealing with custom fonts, noise, or plate layouts unique to Egypt. Therefore, we conducted a **fine-tuning** process to improve its accuracy and reliability for our application.

Table 7-3 Easy OCR for License Plate Recognition

Item	Value
Base Model	EasyOCR
Pre-trained on	Mixed-language OCR Dataset

Fine-Tuning Process

The fine-tuning of EasyOCR involved retraining the model on a custom dataset of **cropped license plate** images, extracted from our vehicle image dataset using the YOLO V11 detection outputs. Each cropped region was labeled with its correct text (in Arabic numerals and letters), allowing the model to learn the exact visual structure of Egyptian license plate content.

The fine-tuned EasyOCR model was evaluated on a validation set containing Arabic license plates captured under varied lighting and environmental conditions. The results showed a notable improvement in both character-level and full-plate recognition accuracy compared to the default pretrained model.

Below is a summary of **the character-level performance**, highlighting commonly used Arabic letters and numerals:

Results of Fine-Tuning OCR

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
12/12 [00:24:00:00, 2.05s/it]						
all	745	2894	0.979	0.989	0.989	0.662
0	107	116	0.93	0.922	0.941	0.403
1	214	309	0.971	0.99	0.988	0.544
2	152	199	0.991	0.99	0.988	0.626
3	137	186	0.982	0.995	0.985	0.665
4	144	208	0.99	0.997	0.993	0.629
5	127	204	0.979	0.995	0.995	0.621
6	155	244	0.979	0.973	0.993	0.667
7	139	189	0.998	0.995	0.995	0.681
Zaa	48	49	0.986	1	0.995	0.669
8	151	192	0.978	0.99	0.979	0.665
9	132	170	0.986	0.982	0.982	0.621
Taa	61	69	0.993	1	0.995	0.686
Thaa	45	48	0.981	0.979	0.98	0.69
sin	91	108	0.981	0.99	0.99	0.716
alif	90	101	0.971	0.97	0.991	0.559

Figure 7-8 Results of fine tuning of ocr

7.10 Precision & Recall & Avg Precision

- **Precision** measures the proportion of predicted positives that are actually correct.
- $$\text{Precision} = \frac{TP}{TP+FP}$$
- **Recall** measures the proportion of actual positive that were predicated correctly
- $$\text{Recall} = \frac{TP}{TP+FN}$$

- **Average precision (AP)** is a popular metric in measuring the accuracy of object detectors like YOLO
- AP → Calculated for each class alone
- **Mean Average precision (mAP)** is the average of (AP) for all detector classes
- $mAP = 1/n * \sum (AP)$

7.11 Evaluation

7.11.1 Training and Evaluation of Yolo

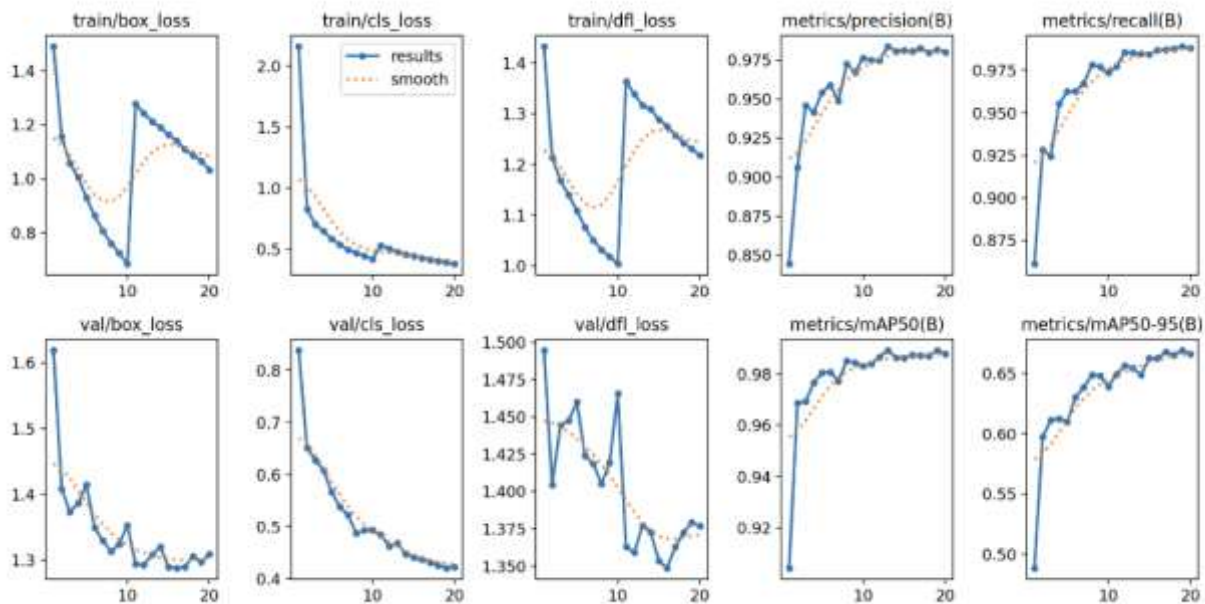


Figure 7-9 Evaluation of Yolo

- train/box_loss: Dropped from ~1.45 to ~0.75 → better bounding box fitting over time.
- val/box_loss: Decreased from ~1.6 to ~1.3 and stabilized → shows good generalization.
- train/cls_loss: Decreased smoothly from ~2.05 to ~0.45 → improved classification during training.
- val/cls_loss: Dropped from ~0.83 to ~0.43 → consistent validation classification performance.
- train/dfl_loss: Declined from ~1.45 to ~1.2 → improved box distribution predictions.
- val/dfl_loss: Slight drop from ~1.49 to ~1.37, with small fluctuations → acceptable stability.
- metrics/precision: Increased steadily from ~0.86 to ~0.975
- metrics/recall: Improved from ~0.91 to ~0.98
- metrics/mAP@50: Climbed from ~0.91 to ~0.985
- metrics/mAP@50-95: Rose from ~0.50 to ~0.675

7.11.2Evaluation of OCR

7.11.2.1 Confusion Metrics of OCR

The raw confusion matrix shows the actual number of correct and incorrect predictions made by the model for each character. Most of the values are concentrated along the diagonal, indicating that the model was able to correctly predict the majority of characters. The presence of off-diagonal values reflects some level of misclassification, which is expected in real-world scenarios.

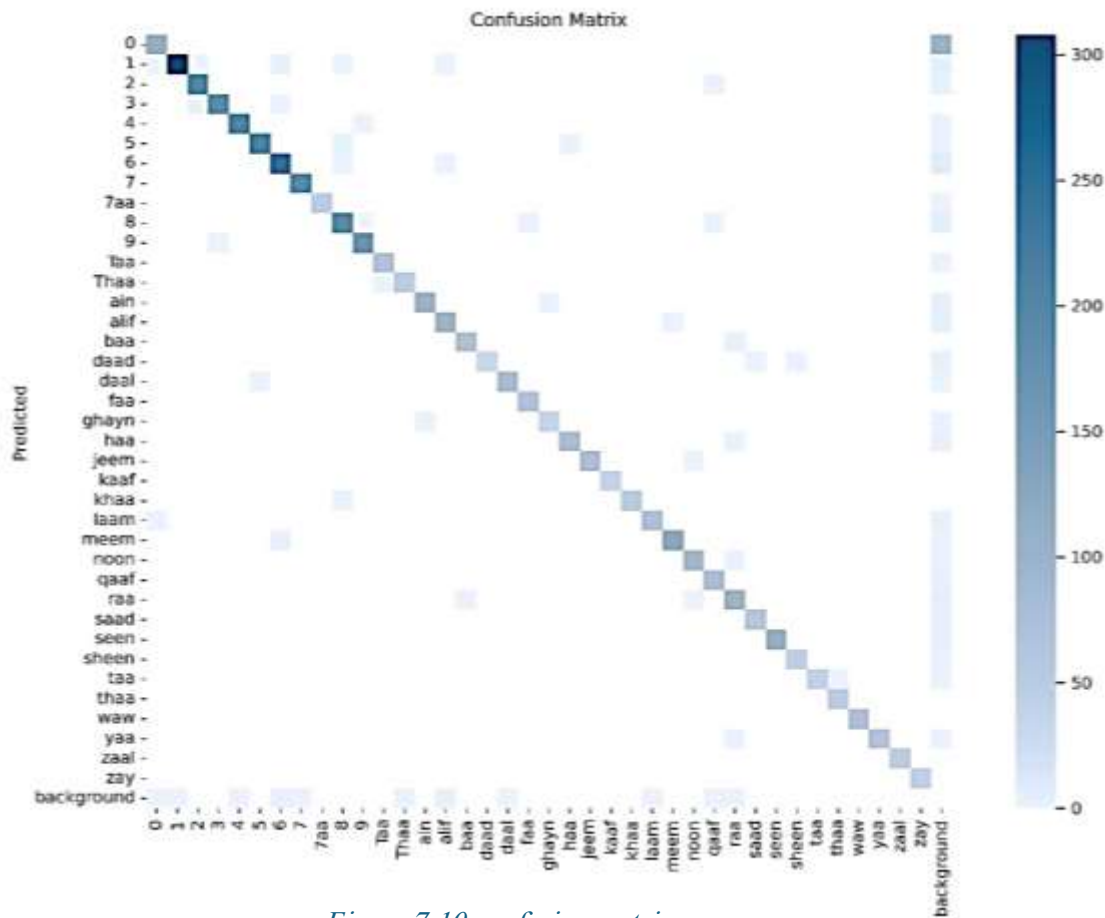


Figure 7-10 confusion matrix

7.11.2.2 Normalized Confusion Matrix

The normalized confusion matrix represents the per class prediction accuracy as a percentage. Each row sums to 1, showing how often the model predicted each character correctly relative to the total predictions for that character. The diagonal dominance suggests that the model generalizes well across the dataset, while lighter values off the diagonal point to occasional confusion between certain characters



Figure 7-11 Normalized Confusion matrix

7.11.2.3 Overall Evaluation Summary

Based on the training and validation results of both YOLOv11 and EasyOCR models, it is clear that the models have successfully learned the patterns in the dataset.

- YOLOv11 showed high detection accuracy with minimal overfitting, as indicated by the close performance between training and validation loss.
- EasyOCR also demonstrated strong recognition performance, especially after fine tuning, with consistent results across both training and unseen validation images.

These results confirm that the models generalized well and are reliable for real-world Egyptian license plate recognition scenarios.



Figure 7-12 evaluation summarized

Chapter 8

System Evaluation & Testing

8 Chapter 8

System Evaluation & Testing

8.1 Introduction

After completing the construction of the project, it is essential to ensure that the project is running efficiently and that there are no problems that can cause danger, especially when it comes to people's safety and security. Door locks are one of the most delicate and complex matters to deal with. Section 8.1 presents some system test scenarios and how the system works.

System Testing Scenarios

The Test Objective is a clear statement about what the test aims to achieve (e.g., verifying login functionality, testing system performance under load).

8.2 Test Case Points

- **Index**
 - A unique identifier for the test case.
- **Test Case**
 - It is a description of the feature based on which it is tested to find out whether it works or not
- **Test step**
 - A detailed list of actions the tester needs to perform to execute the test.
- **Expected Result**
 - The anticipated outcome of the test if everything functions correctly.
- **Actual Result**
 - The outcome observed when the test is executed.
- **Status**
 - Whether the test passed or failed

This document outlines various testing scenarios to comprehensively evaluate the functionality, security, and performance of the automatic parking system. The testing scenarios are shown as test cases of the hardware in **table 8.1**

Table 8-1 System Testing

Index	Test Name	Test Case	Precondition	Test Steps	Test Data	Output
1	Login_1	Verification of login page with valid username and Password	Username and password need to match for login	1) type correct username. 2) type correct password. 3) press on the login button.	1) valid username. 2) valid password. 3) button clicks.	Login successful
2	Login_2	Verification of login page with invalid username and valid Password	Username and password need to match for login	1) type incorrect username. 2) type correct password. 3) press on the login button.	1) invalid username. 2) valid password. 3) button clicks.	<u>Error:</u> This username you entered for the username is incorrect.

3	Login_3	Verification of login page with valid username and invalid Password	Username and password need to match for login	1) type incorrect username. 2) type correct password. 3) press on the login button.	1) valid username. 2) invalid password. 3) button clicks.	Error: This password you entered is incorrectly.
4	Creditcard_1	Verification of Credit card with valid Credit card number and Expiration date and CVV	Credit card number and Expiration date and CVV need to be correct	1) type correct Credit card number. 2) type correct Expiration date. 3) type correct CVV. 4) press on the Make Reservation button	1) valid Credit card number. 2) valid Expiration date. 3) valid CVV. 4) button clicks.	Reservation successful
5	Creditcard_2	Verification of Credit card with invalid Credit card number and valid Expiration date and valid CVV	Credit card number and Expiration date and CVV need to be correct	1) type incorrect Credit card number. 2) type correct Expiration date. 3) type correct CVV. 4) press on the Make Reservation button.	1) invalid Credit card number. 2) valid Expiration date. 3) valid CVV. 4) button clicks.	<u>Error:</u> This Credit card number you entered is incorrectly.
6	Credit card_3	Verification of Credit card with valid Credit card number and invalid	Credit card number and Expiration date and CVV need to be correct	1) type correct Credit card number. 2) type incorrect	1) valid Credit card number. 2) invalid Expiration date.	<u>Error:</u> This Expiration date you entered is incorrectly.

		Expiration date and valid CVV		Expiration date. 3) type correct CVV. 4) press on the Make Reservation button.	3) valid CVV. 4) button clicks.	
7	Credit card_4	Verification of Credit card with valid Credit card number and valid Expiration date and invalid CVV	Credit card number and Expiration date and CVV need to be correct	1) type correct Credit card number. 2) type correct Expiration date. 3) type incorrect CVV. 4) press on the Make Reservation button.	1) valid Credit card number . 2) valid Expiration date. 3) invalid CVV. 4) button clicks.	<u>Error:</u> This CVV you entered is incorrectly.
8	Payment_1	Verification of payment with valid method	Payment needs to be made	1) scan RFID tag	1) valid payment	Open Exit Gate.
9	Payment_2	Verification of payment with Invalid method	Payment needs to be made	1) scan RFID tag	1) invalid payment	<u>Error:</u> This Payment is valid (repeat payment)

Chapter 9

Parking Reservation System Market

9 Chapter 9

Parking Reservation Market

9.1 Introduction

The Parking Reservation System Market is a rapidly growing sector that addresses the increasing demand for efficient and convenient parking solutions. This market encompasses technologies and services that enable users to easily find, reserve, and pay for parking spaces, often through mobile applications or online platforms.

9.2 Market

The Global Parking Reservation System market size reached USD 1.23 billion in 2023 and is expected to reach USD 4.74 billion by 2030. Parking Reservation System, growing at a CAGR of 21.2 % during the forecast period. The growing urban population has taken shape as a major problem with the parking system in almost every major city around the world. Many of us are very disturbed when there is not enough parking space for our vehicles. People can eradicate this problem by implementing an online Car parking reservation system. They

can find and book a parking spot by using their smartphones. Airlines are among the most regular users of the online parking reservation system, with 90% of UK airports having an online Parking Reservation service, while implementation at airports in other European countries increases by 30% every year. The rising trend of smart cities throughout the world is driving demand for smart parking solutions, which is supporting the Parking Reservation System market growth during the forecast period. Some established parking businesses as well as new technological entrants are already looking to capitalize on these developments by creating new and innovative ways to control demand, improve supply, and better meet the

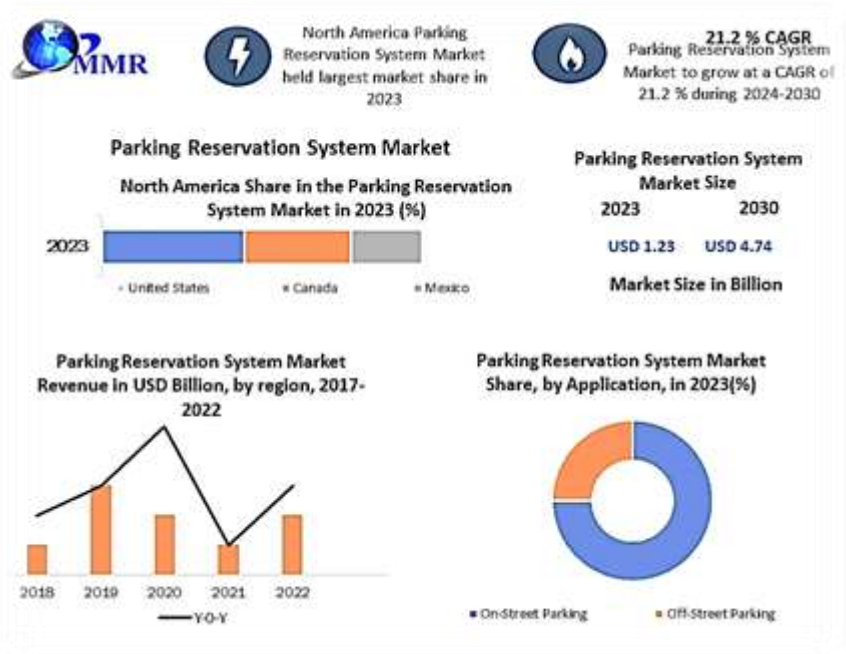


Figure 9-1 Market Trend

demands of consumers. VINCI Park S.A., Street line, Inc., National Car Parks Ltd., APCOA Parking AG, Ace Parking Management, Inc.,

9.2.1 Target Audience

Identifying the right target audience is essential for the marketing strategy. The primary target market segments for an advanced automatic parking system **include:**

- 1) **Commercial Property Owners:** Individuals and businesses that own parking lots, garages, or commercial properties. **This includes:**
 - **Retail centers:** Malls, shopping centers, and strip malls.
 - **Office buildings:** Corporate offices, commercial office parks, and business centers.
 - **Hospitals and healthcare facilities:** Medical centers, clinics, and hospitals.
 - **Hotels and resorts:** Lodging establishments.
 - **Stadiums and arenas:** Sports venues.
 - **Convention centers:** Event venues.
- 2) **Residential Property Owners:** Individuals who own residential properties with limited parking spaces, such as condominiums, apartment complexes, and gated communities.
- 3) **Municipalities and Cities:** Local governments that manage public parking facilities, such as on-street parking, municipal parking lots, and parking garages.

By targeting these specific segments, we can effectively market the benefits of an advanced automatic parking system, such as increased efficiency, reduced congestion, and enhanced security.

9.3 Product Positioning

Effective product positioning highlights the unique value proposition and differentiates the automatic parking system from competitors. The advanced automatic parking system will be positioned as:

- **Innovative and Reliable:** Utilizing state-of-the-art technology, including advanced sensors, AI, and IoT, to provide seamless and efficient parking experiences.
- **User-Friendly:** Easy-to-use mobile app and intuitive interface for a hassle-free parking experience.

- **Scalable:** Flexible solutions for various parking scenarios, from small residential complexes to large-scale commercial facilities.
- **Cost-Effective:** Long-term savings through increased efficiency, reduced labor costs, and optimized space utilization.
- **Eco-Friendly:** Energy-efficient solutions that promote sustainability and reduce carbon footprint.
-

9.4 Promotional Strategies

To create awareness and generate interest in the automatic parking system, a multi-faceted promotional strategy will be employed, **including:**

1. Digital Marketing:

- **Social Media Campaigns:** Leveraging platforms like Facebook, Instagram, LinkedIn, and Twitter to reach potential customers through targeted ads and engaging content.
- **Search Engine Optimization (SEO):** Optimizing the Application to rank higher in search engine results for relevant keywords like "parking system," "smart parking," and "parking solutions."
- **Pay-Per-Click (PPC) Advertising:** Running targeted ads on Google Ads and other search engines to drive traffic to the website and generate leads.
- **Email Marketing:** Sending targeted email campaigns to potential customers, existing clients, and industry professionals.

2. Content Marketing

- **Blog Posts:** Publishing informative articles on the benefits of automatic parking systems, industry trends, and case studies.
- **Whitepaper:** Creating in-depth reports on specific topics, such as the impact of smart parking on urban mobility.
- **Webinars and Seminars:** Hosting online and in-person events to educate potential customers and industry professionals.

3. Public Relations

- **Press Releases:** Announcing product launches, partnerships, and company milestones to media outlets.
- **Media Outreach:** Building relationships with journalists and industry influencers to secure media coverage.
- **Crisis Communication:** Developing a crisis communication plan to manage potential negative publicity.

4. Partnerships and Collaborations

- **Strategic Partnerships:** Collaborating with technology providers, real estate developers, and municipalities to expand market reach.

• **Channel Partnerships:** Partnering with distributors and resellers to increase sales and market penetration.

5. Sales Promotions

- **Discounts and Offers:** Providing limited-time discounts and special offers to attract new customers.
- **Loyalty Programs:** Implementing loyalty programs to reward repeat customers.
- **Referral Programs:** Encouraging customers to refer friends and family by offering incentives.

9.5 Conclusion

Implementing a comprehensive marketing strategy is vital for the successful launch and adoption of the automatic parking system. By leveraging market insights, targeting the right audience, positioning the product effectively, and employing diverse promotional tactics, the project aims to establish a strong market presence and drive significant sales growth.

The parking reservation market is undergoing significant transformation driven by the rise of smart city initiatives, increased urban congestion, and the widespread adoption of mobile and digital technologies. Consumers are increasingly demanding real-time, convenient, and efficient parking solutions, prompting growth in app-based and cloud-powered reservation platforms. The market is expected to expand steadily, with commercial zones, airports, and event venues serving as primary growth areas. However, challenges such as data privacy concerns, integration with existing infrastructure, and regulatory compliance must be

addressed. Overall, the parking reservation market holds strong potential, particularly as sustainability and smart mobility continue to shape the future of urban transportation.

Chapter 10

Conclusion and Future Work

10 Chapter 10

Conclusion and future work

10.1 Conclusion

The successful implementation of this Automatic Parking System, powered by ESP32 and ESP32 CAM, marks a significant stride towards the future of smart cities. By seamlessly integrating RFID technology, mobile applications, and advanced security features, this system has redefined the parking experience.

Initially, we implemented the system using ESP32, focusing on integrating a keypad and RFID reader for access control to enter the parking system and LCD For display the empty and full places according to sensors in the parking and fire system to detect if there is a fire in parking and temperature sensor for help fire sensors if fail. This phase allowed us to establish a solid foundation for the system, ensuring that essential components such as the keypad input, RFID scanning, LCD, IR sensor and Servomotor mechanism functioned correctly and reliably.

The transition to ESP32 marked a significant enhancement in our project. The ESP32's built-in Wi-Fi and Bluetooth capabilities introduced the potential for remote access and control, significantly improving the system's flexibility and convenience. By designing and fabricating a custom PCB for the ESP32, we achieved a more compact and efficient hardware configuration, reducing the overall footprint and power consumption of the system.

in conclusion, this Automatic Parking System project showcases the successful integration of advanced microcontroller technology, specifically ESP32, with cutting-edge IoT concepts. By combining RFID technology, mobile applications, and robust security features, this system has redefined the parking experience.

The system's ability to automate parking processes, streamline payments, and enhance security has not only improved user convenience but also optimized resource utilization. The integration of fire detection and temperature monitoring systems further underscores commitment to safety and sustainability. This project serves as a testament to the potential of technology to address real-world challenges and improve urban living. As we continue to explore the possibilities of smart city technologies, we envision a future where parking is no longer a hassle but a seamless and efficient experience.

10.2 Future Work

To further enhance the capabilities of the Automatic Parking System, several avenues for future research and development have been identified:

1. Advanced Sensor Integration

- Incorporating advanced sensors, such as ultrasonic sensors or LiDAR, to improve vehicle detection accuracy and parking space allocation.
- Implementing real-time traffic monitoring and occupancy sensing to optimize parking guidance and reduce congestion.

2. AI and Machine Learning

- Utilizing AI and machine learning algorithms to predict parking demand, optimize parking space utilization, and personalize user experiences.
- Developing predictive maintenance models to identify potential system failures and schedule preventive maintenance.

3. Enhanced Security Features

- Integrating biometric authentication, such as fingerprint or facial recognition, for added security.
- Implementing advanced security protocols, such as encryption and secure communication channels, to protect sensitive data.

4. Integration with Smart City Infrastructure

- Integrating the parking system with other smart city components, such as traffic management systems and public transportation, to create a seamless and efficient urban mobility ecosystem.

5. User Experience Optimization

- Continuously refining the mobile application to provide a more intuitive and user-friendly interface.
- Implementing personalized features, such as customized parking preferences and real-time notifications.

10.3 Integration with Smart Home Systems

Future iterations of the Automatic Parking System could focus on seamless integration with existing smart home ecosystems. This would allow users to control various aspects of the system, such as remote parking, payment, and security features, through popular smart home platforms like Google Home, Amazon Alexa, and Apple HomeKit.

Such integration would provide users with a unified interface for managing all smart devices within their home, increasing convenience and interoperability. By connecting the parking system to other smart home devices, users could automate tasks like triggering home lighting or climate control upon arrival or departure.

10.4 Enhanced Security Protocols

To address potential vulnerabilities, future work should focus on implementing advanced encryption techniques and security protocols. This includes end-to-end encryption of communication between the payment and user devices, as well as regular security updates to protect against emerging threats. Additionally, incorporating machine learning algorithms to detect and respond to unauthorized access attempts in real-time could significantly improve the system's defense mechanisms.

10.5 Battery Efficiency and Alternative Power Sources

Improving the battery life of the automatic parking system is crucial for its reliability. Future work could investigate low-power consumption components and optimization of the system's software to extend battery life. Additionally, exploring alternative power sources such as solar panels or kinetic energy harvesting could provide sustainable power solutions and reduce dependency on traditional batteries.

10.6 Testing and Optimization for Diverse Environments

Further testing and optimization are needed to ensure the automatic parking system performs reliably across a wide range of environmental conditions. This includes temperature variations, humidity, and exposure to dust or moisture. Developing robust housing for parking that can withstand harsh conditions would be essential for broadening the system's application in different settings.

10.7 Scalability and Network Capabilities

Exploring the scalability of the system for larger installations, such as buildings or apartment complexes, could open new market opportunities. This would involve developing network capabilities that allow multiple parking to be managed centrally, providing a cohesive security solution for people.

10.8 User Interface and Experience Enhancements

Improving the user interface and overall user experience is an ongoing priority. Future iterations should focus on refining the mobile application's interface, making it more intuitive and user-friendly.

10.9 Legal and Ethical Considerations

As the Automatic Parking System evolves, it is crucial to address legal and ethical considerations related to privacy and data protection. Future work should prioritize the following:

- **Data Privacy Compliance:** Ensuring compliance with relevant data protection regulations, such as GDPR and CCPA.
- **Transparent Data Practices:** Developing clear and transparent data usage and retention policies.
- **Ethical Data Handling:** Implementing robust security measures to protect user data and prevent unauthorized access.
- **User Consent:** Obtaining explicit user consent for data collection and processing.

By addressing these ethical considerations, we can build trust with users and ensure the responsible development and deployment of the Automatic Parking System

11 References

- [1] A. Quadri, A. Kumar, T. Sahu, and P. Kumar, "Advanced Control Strategies for Automatic Parking Systems in Smart Cities," *Int. J. Autom. Control Syst*, Jun. 2023.
- [2] A. Al Misbah, M. Al-Hashim, and S. Elnakla, "Sensor Fusion and Path Planning for Autonomous Parking Vehicles," *IEEE Trans. Veh. Technol*, May 2025.
- [3] P. Kumar and R. Singh, "IoT based multi-level automatic parking system," in *Proc. 2019 6th Int. Conf. Internet of Things: Smart Innovation and Usages (IoT-SIU)*, Ghaziabad, India, Mar. 2019, pp. 1–6.
- [4] F. Youssif, F. Gamil, and E. S. A. Bakkar, "Intelligent Controller Design for a Novel Automatic Parking System," Jun 2023
- [5] T.-Y. Lee and C.-F. Lee, "Microcontroller based automatic parking system," in *Proc. 2012 Int. Conf. Machine Learning and Cybernetics (ICMLC)*, Xian, China, Jul. 2012, pp. 875–879
- [6] M. Meng, H. Guo, J. Xu, and D. Lu, "Design of automatic parking system," in *Proc. 2011 Int. Conf. Electrical and Control Engineering (ICECE)*, Yichang, China, Sep. 2011, pp. 6440–6443.
- [7] N. Bibi, M. N. Majid, H. Dawood, and P. Guo, "Automatic Parking Space Detection System," in *Proc. 2017 2nd Int. Conf. Multimedia and Image Processing (ICMIP)*, Wuhan, China, Mar. 2017, pp. 7–11.
- [8] N. Aphiratsakun, L. T. Duy, and J. Kongthon, "AU Automatic Car-Parking Modeling System," in *Proc. 2018 Int. Electrical Eng. Congress (iEECON)*, Krabi, Thailand, Mar. 2018, pp. 1–4.
- [9] C.-Y. Wu, S.-H. Huang, P.-L. Peng, and G.-J. Jong, "Automatic vehicle parking system using gyroscope," in *Proc. 3rd Int. Conf. Ubiquitous Information Management and Communication (ICUIMC)*, Suwon, South Korea, Jan. 2009, pp. 1–5.
- [10] K. An, J. Choi, and D. Kwak, "Automatic Valet Parking System incorporating a nomadic device and parking servers," in *Proc. 2011 IEEE Int. Conf. Consumer Electronics (ICCE)*, Las Vegas, NV, USA, Jan. 2011, pp. 883–884.

- [11] H.-C. Huang, Y.-H. He, and F. Lin, "A vehicle transverse automatic parking auxiliary system," in Proc. 2015 Int. Conf. Machine Learning and Cybernetics (ICMLC), Guangzhou, China, Jul. 2015, pp. 1273–1278.
- [12] S. Gatha, D. Pratiksha, K. Pranita, I. Sneha, and S. Rudrawar, "Automatic Car parking system using Google Assistant," in Proc. 2021 Int. Conf. Communication Information and Computing Technology (ICCICT), Mumbai, India, Jun. 2021, pp. 1–4
- [13] C. H. Chen, C. W. Hsu, and C. C. Yao, "A novel design for full automatic parking system," in Proc. 2012 12th Int. Conf. ITS Telecommunications (ITST), Taipei, Taiwan, Nov. 2012, pp. 1–5.
- [14] I. H. Chowdhury, A. Abida, and M. M. H. Muaz, "Automated vehicle parking system and unauthorized parking detector," in Proc. 2018 20th Int. Conf. Advanced Communications Technology (ICACT), Chuncheon, South Korea, Feb. 2018, pp. 1–6.

Reference for AI Model

[1] Amany Sarhan, Shahd Hatem, Bassel Darwish Deep machine learning based Egyptian vehicle license plate recognition systems. In: Al-Azhar engineering 15th international conference, Egypt

From:

https://www.researchgate.net/publication/255964716_License_Plate_Recognition_System_For_Egyptian_Car_Plates

[2] Al hyaline S (2022) Real-time Jordanian license plate recognition using deep learning. J King Saud Univ Computer Inf Sci 34:6

From: <https://link.springer.com/article/10.1007/s00521-025-11302-6>

[3] Youssef AR, Ali AA, Sayed FR (2022) Real-time Egyptian license plate detection and recognition using YOLO. Int J Adv Computer Sci Appl 13(7):66

From: <https://www.sciencedirect.com/science/article/pii/S2090447923000679>

[4] Sarhan et al., "Egyptian car plate recognition based on YOLOv8, Easy-OCR, and CNN," *Journal of Electrical Systems and Information Technology*, vol. 11, no. 32, 2024.

الملخص

أدت الأنشطة الاقتصادية المتزايدة في المناطق الحضرية إلى ارتفاع كبير في حركة المركبات، مما أدى إلى مشكلة الازدحام المروري الملحة. أحد التحديات الرئيسية التي يواجهها أصحاب المركبات في هذه المدن المزدهمة هو ندرة مواقف السيارات، وخاصة خلال ساعات الذروة. إن نقص المعرفة بشأن توفر مواقف السيارات في أي وقت من الأوقات يزيد من تفاقم هذه المشكلة. لمعالجة هذه التحديات، تقدم هذه الوثيقة حلاً مبتكراً.

نظام مواقف سيارات آلي وذكي. يهدف نظام مواقف السيارات الآلي المقترح إلى تخفيف الازدحام المروري في المناطق الحضرية من خلال توفير تجربة ركن سيارات فعالة وسهلة الاستخدام. باستخدام أزواج من أجهزة الإرسال والاستقبال بالأشعة تحت الحمراء، ينقل النظام عن بُعد حالة إشغال موقف السيارات إلى وحدة تحكم ESP32. ثم تُعرض هذه المعلومات على لوحة عرض بارزة تقع عند مدخل موقف السيارات. من خلال هذه الشاشة، يمكن لأصحاب المركبات التأكد فوراً من توفر أو عدم توفر مواقف السيارات قبل دخول موقف السيارات، مما يوفر لهم وقتاً وجهداً ثمينين.

ومن المأمول أن يساهم تطبيقه في مواقف السيارات في حل المشكلات التي يواجهها رواد المواقف. يعتمد نظام مواقف السيارات الذكي على متحكم دقيق من نوع ESP 32 عادةً، في مواقف السيارات الكبيرة، نحتاج إلى البحث في جميع أنحاء المكان عن مساحة ثم ركن السيارة. يهدف مشروعنا إلى التغلب على هذه المشكلة. نريد وضع بعض المستشعرات في مساحة لكل سيارة. سيتم توصيل هذه المستشعرات بشاشات مثل شاشات LCD من خلال المتحكم الدقيق عند مدخل موقف السيارات. سيُظهر العرض ما إذا كانت هناك مساحة كافية لمزيد من السيارات، وإذا وُجدت، فما هو موقع هذه المساحة بالضبط. سيتم تنفيذ ذلك عن طريق البرمجة من خلال متحكم ESP32.

لذا، فإن المشروع يدور حول مواقف عامة لجميع الأشخاص تتكون من مكان دخول السيارة إلى الموقف ومكان آخر للخروج منه، ويتم عرض مخرجات النظام على شاشات LCD للأماكن الخالية للسيارات التي تدخل الموقف، نستخدم لهذا مفتاح زر الضغط، وسنقوم بتوصيله بمتحكم ESP32 للحصول على عدد السيارات في الموقف. ستكون هناك شاشة (LCD) في المكان الذي تدخل منه السيارة لإظهار ما إذا كان الموقف ممتلئاً أو به أماكن لسيارة أخرى، حتى يتمكن الشخص من الذهاب إلى أي مكان فارغ ثم وضع سيارته فيه. في كل مكان في كل موقف سيارة، سيكون لدينا مستشعرات (مفتاح) تُظهر ما إذا كان المكان فارغاً أو متاحاً، وتُظهر شاشة ذلك الموقف ذلك. يتكون النظام من متحكم دقيق وأجهزة استشعار وشاشات (LCD)، وسيتم توصيل جميعها ببعضها البعض في المشروع للحصول على نظام مواقف سيارات ذكي تماماً.

نظام موقف السيارات القائم على المتحكم الدقيق

مشروع تخرج علوم حاسوب

مقدم بواسطة

عبد الله صلاح عبد المجيد أحمد

يوسف أحمد علاء الدين

خالد كرم محمد سليمان

محمد حسين عبد الله حسن

ماريو أمير فكري حنا

يوسف حمدي أمين شمندي

حمزة محمد عبد الله إبراهيم

محمد أيمن أمين علي خليل

يشرف عليها

أ.د. سيد بكار

م.م أحمد علام

م.م مهرائیل کمال

2025